

Introduction to Elasticsearch

Course 2 : Data Aggregation and Visualization in Kibana" in English.

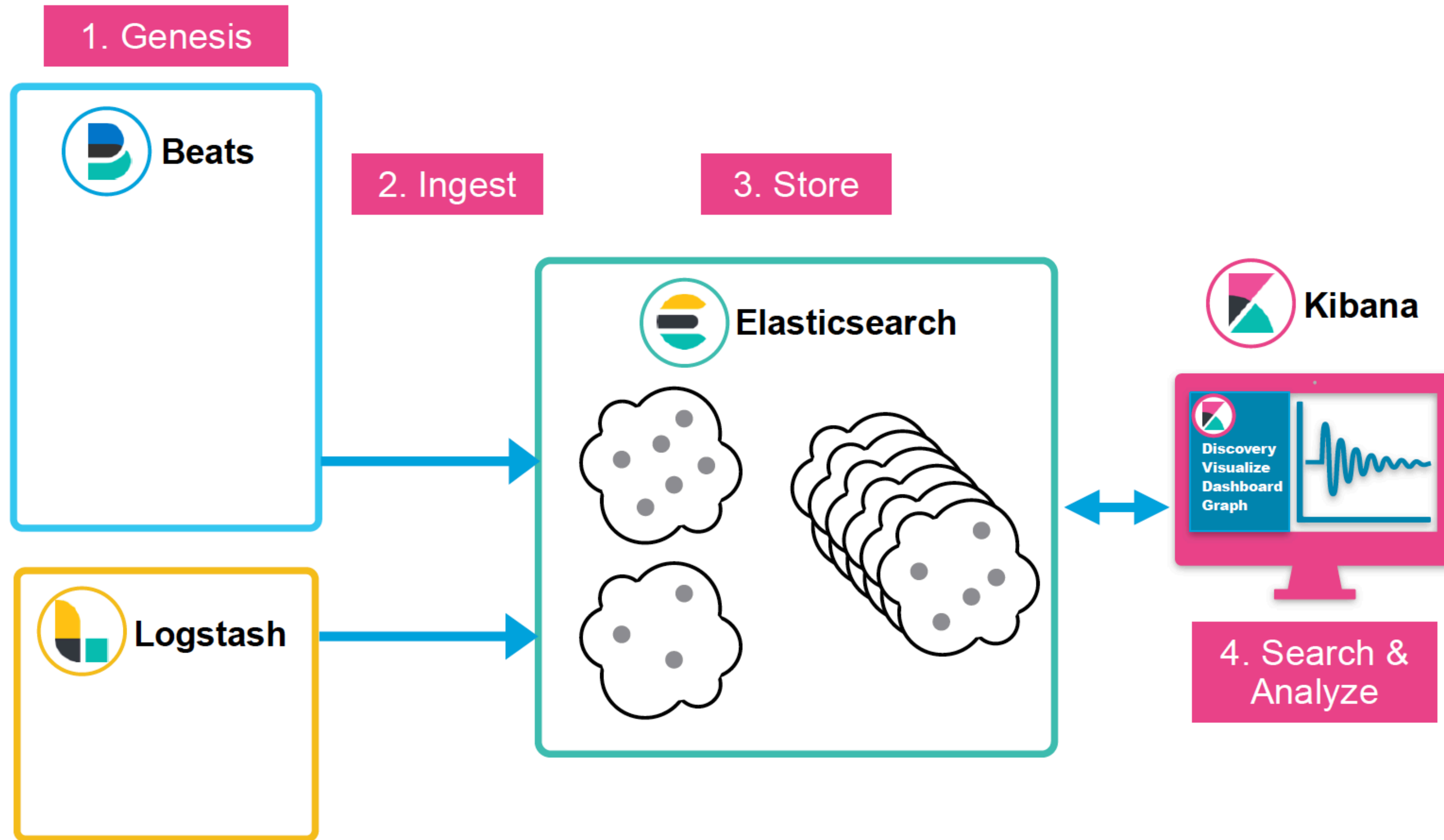
Module 1 : Presentation

- Introduction to Kibana
- Discover Interface
- Visualizing Data



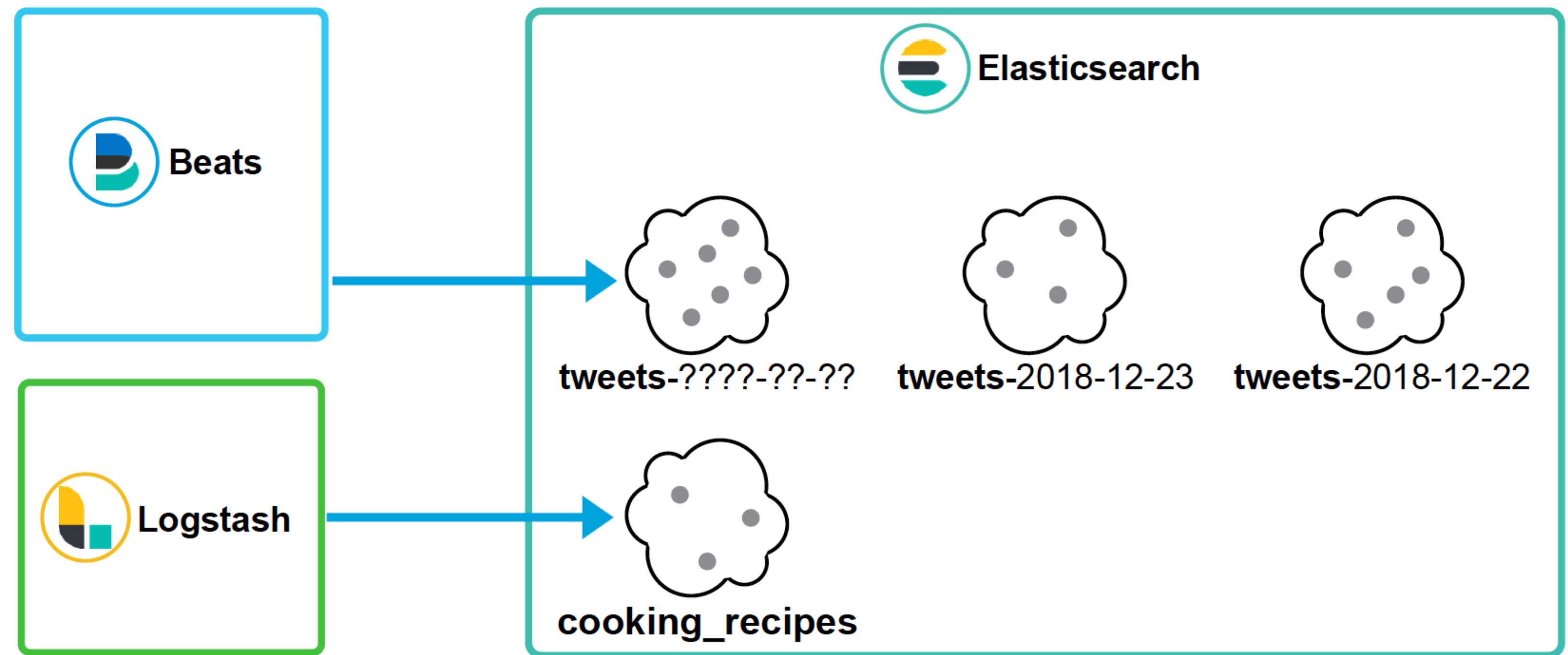
kibana

Data journey



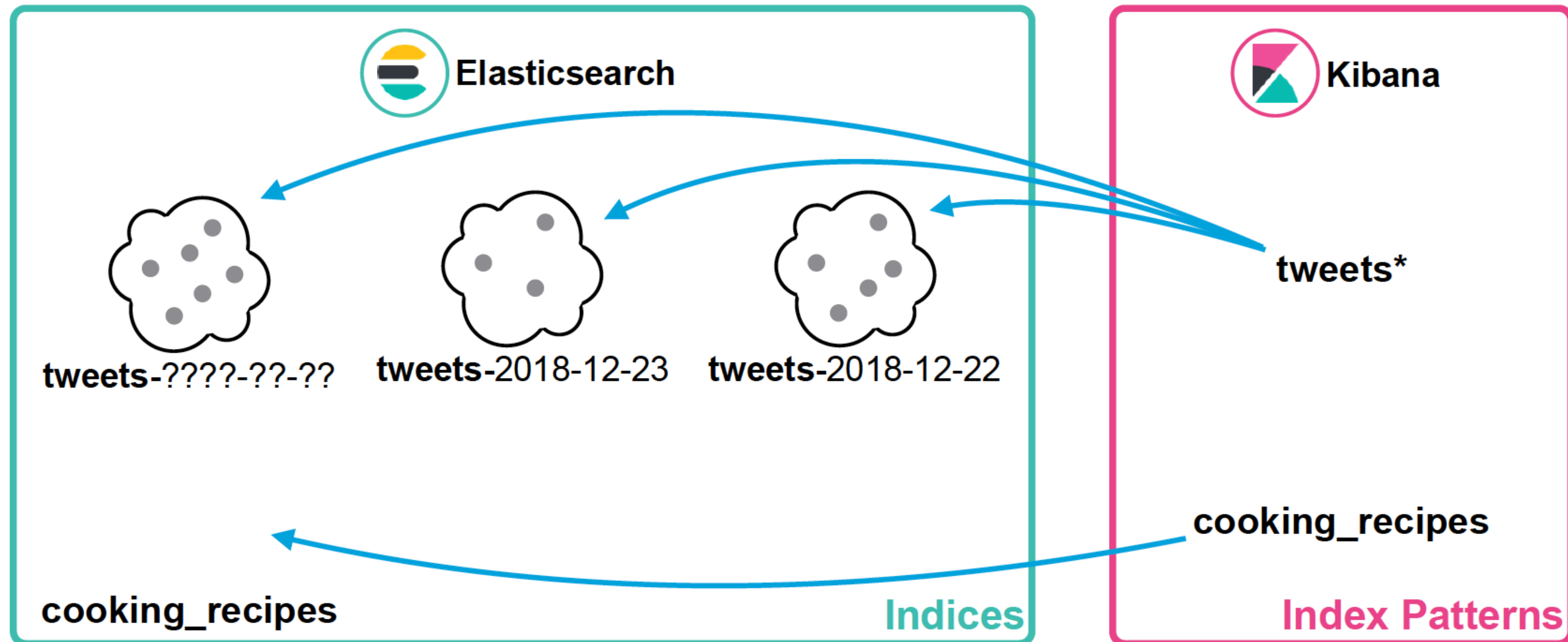
Elasticsearch Index

- Data container
 - categorical index
 - time-based index



Kibana Index Pattern

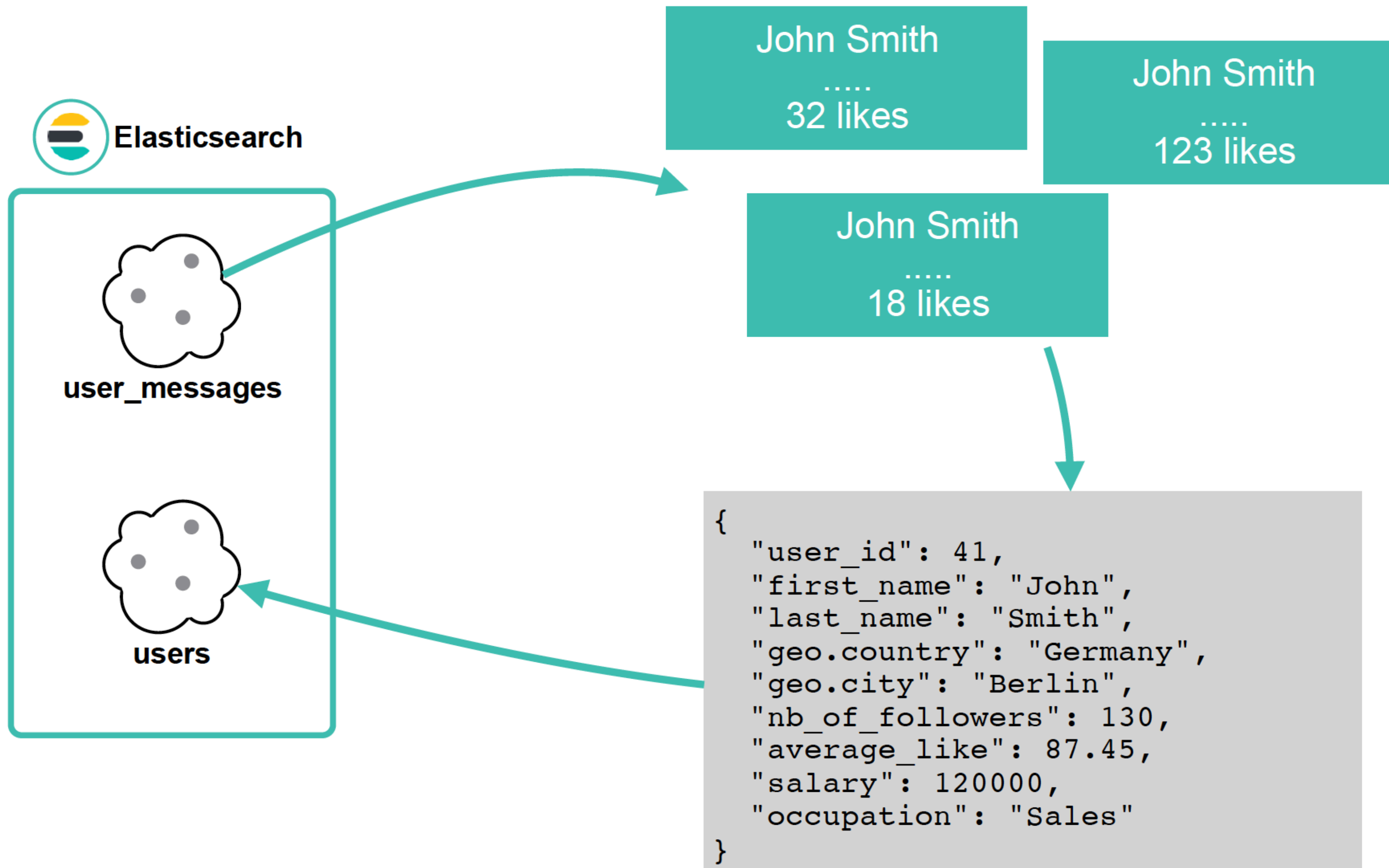
- Points to one or more Elasticsearch indices
- Tells Kibana which data you want to work with



Messages



Users



Data Uploader

- One easy solution to load data in Elasticsearch through Kibana is to use upload feature
- It can be used to load :
 - CSV files
 - log files
 - etc ...
- It will automatically create an index pattern
- It support many advanced features :
 - mapping definition
 - transforming the data

Summary

- Elasticsearch, Kibana, Logstash, and Beats are components of the Elastic Stack
- Kibana can be used to analyze, search, interact with and visualize the data in Elasticsearch
- Kibana can be used to manage the Elastic Stack
- Data is sent as JSON objects into Elasticsearch
- In Kibana, an index pattern can be created to target a specific set of indices

Overview

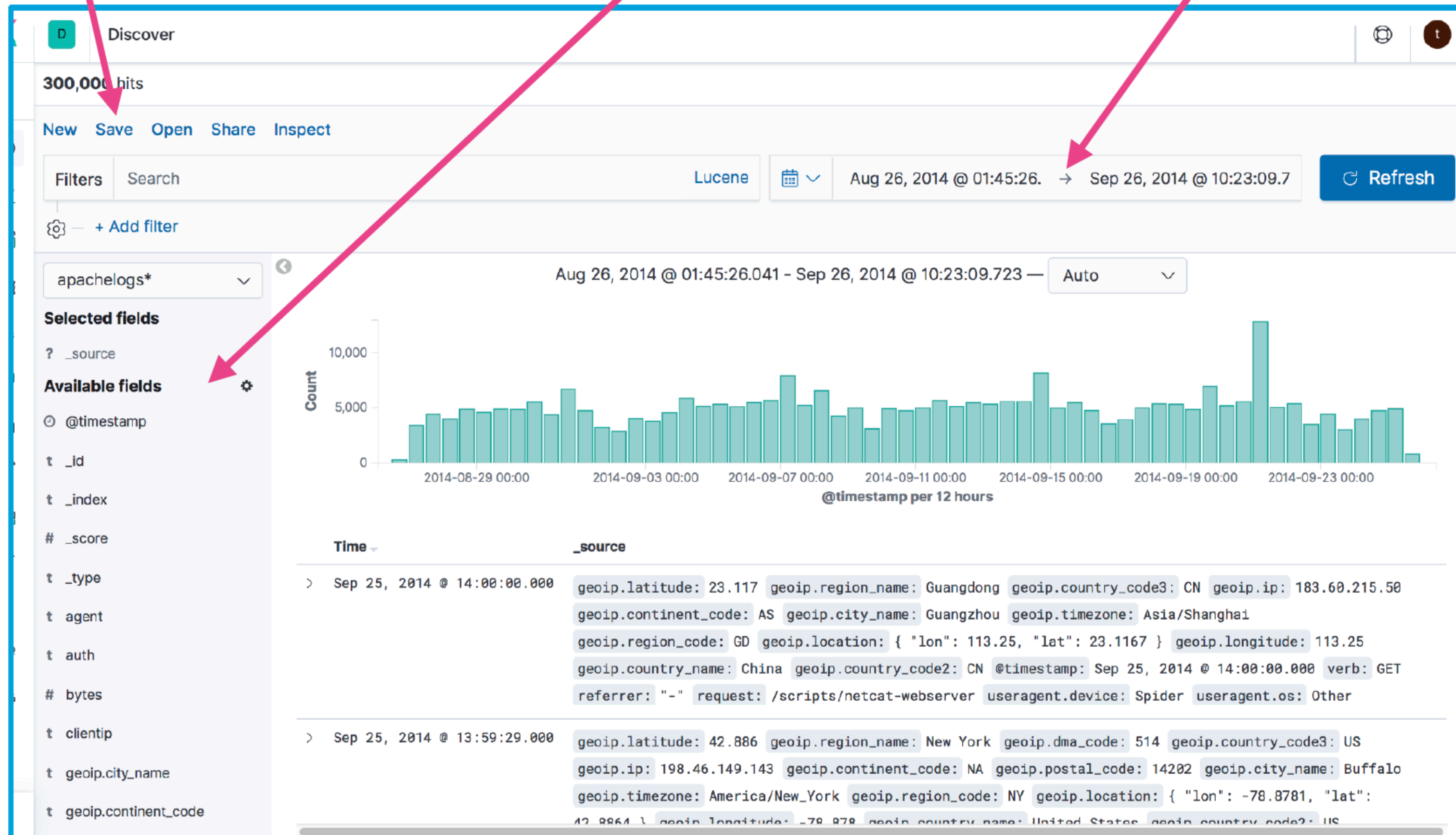
- Elasticsearch data types:
 - numeric
 - text
 - date
 - keywords
 - ...
- Discover interface
 - explore data in Elasticsearch
 - slice and dice (analyze) data

Discover Interface

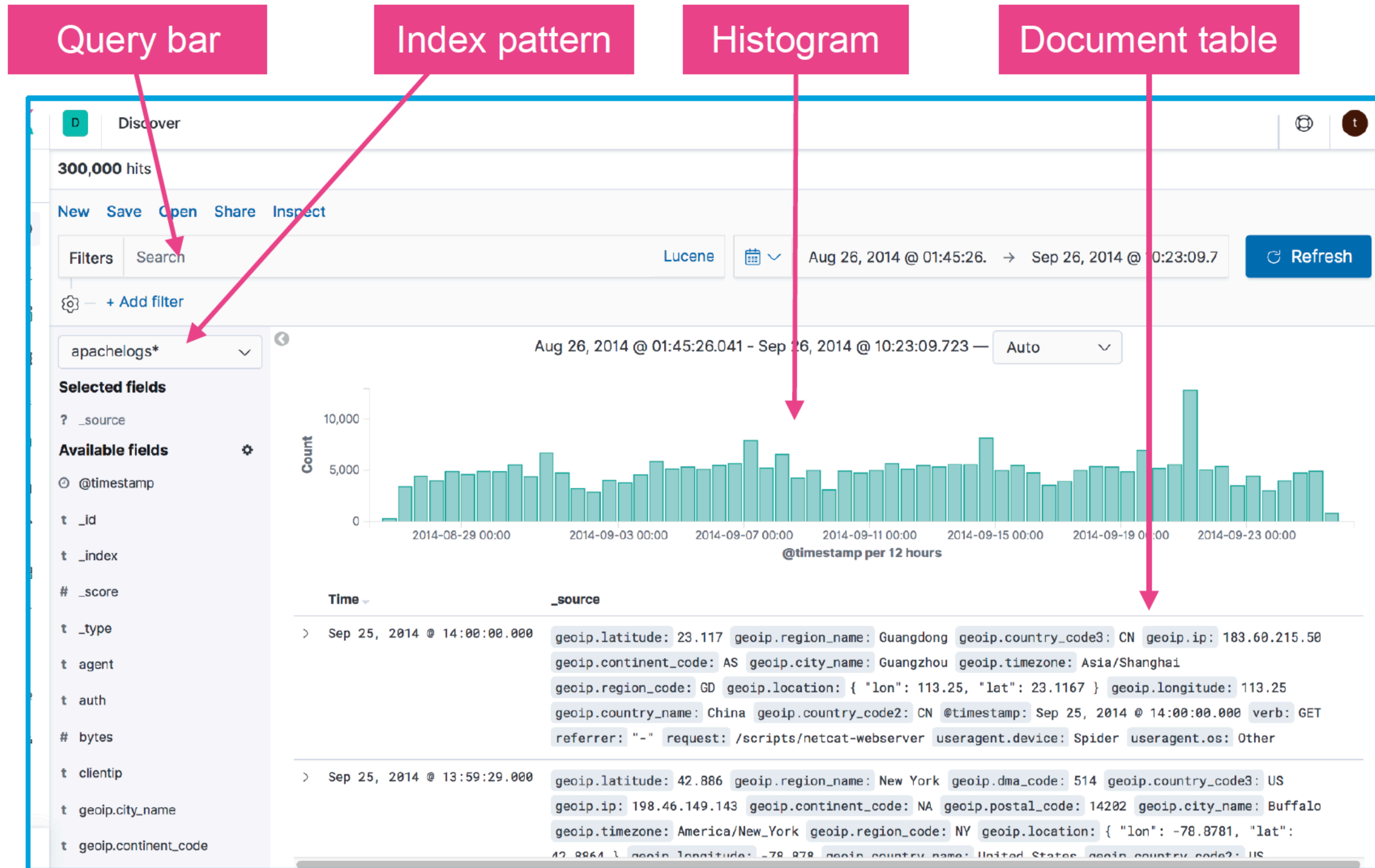
Toolbar

Side navigation

Time picker



Discover Interface



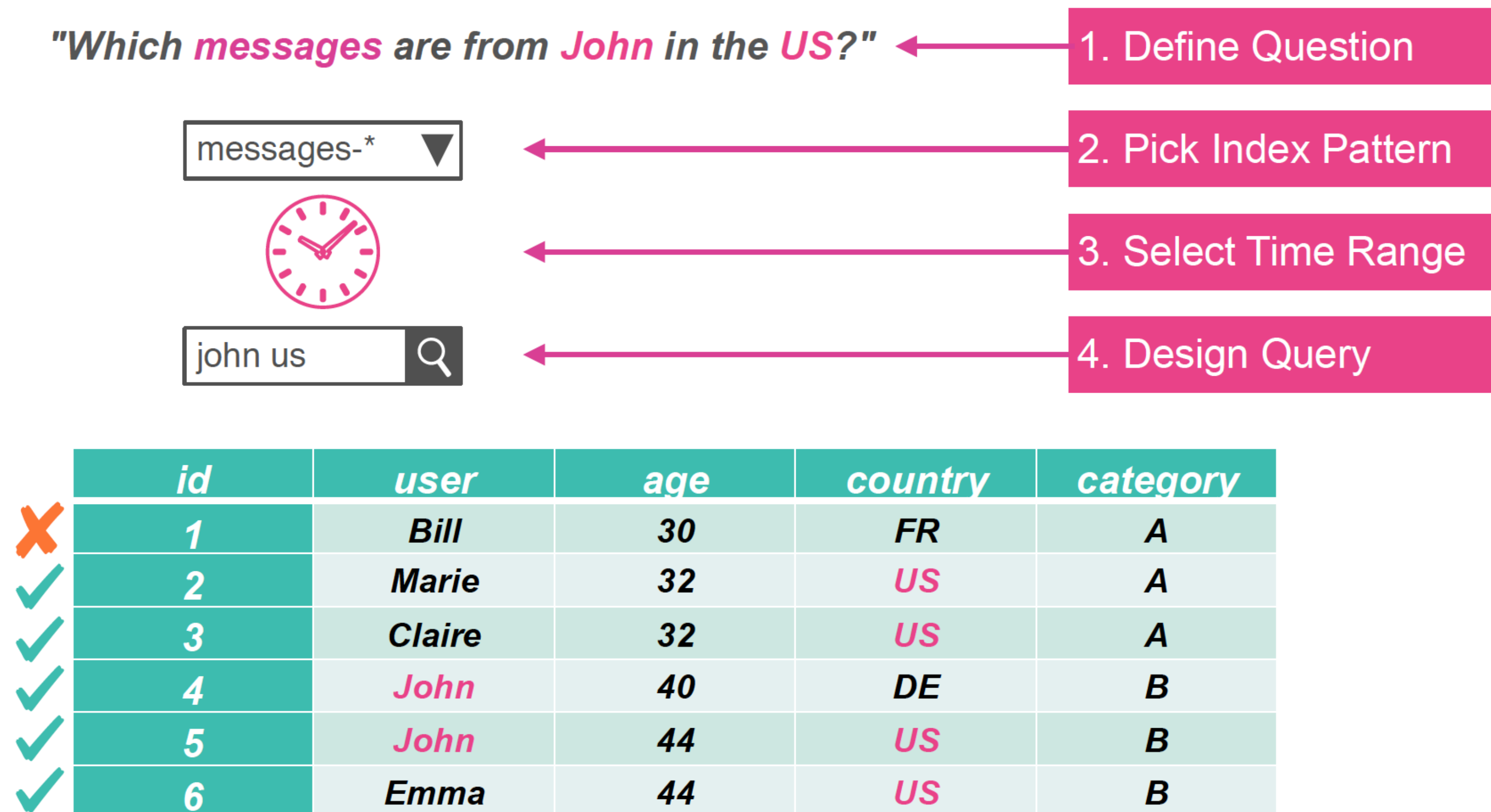
Search is Everywhere

- Elasticsearch is search engine :
 - Kibana can be used to search documents in Elasticsearch
- A search is executed by sending a **query** to Elasticsearch
- In Kibana, a search can be executed from the **query bar**

A horizontal search bar with a light gray border. Inside the bar, on the left, is a small asterisk (*). To the right of the text input area is a dark gray button with a white magnifying glass icon, representing a search function.

Querying

- Kibana supports multiple query languages



Search a Specific Field

- By default, the query below will search all fields for all values

- but being more specific will improve search results

*What are the messages published by **user John** from **country US**?*

- Query above can be made more specific like this :

- Elasticsearch will only need to search limited fields

Boolean Operators

- By default, Kibana uses the **or** logic
 - So it matches any documents containing *john* or *us*
- Kibana allows you to use the following **boolean** operators :
 - **and**, **or**, and **not**
 - Now, you can rewrite the query with the **and** logic

user:john and country:us



	<i>id</i>	<i>user</i>	<i>age</i>	<i>country</i>	<i>category</i>
✗	1	Bill	30	FR	A
✗	2	Marie	32	US	A
✗	3	Claire	32	US	A
✗	4	John	40	DE	B
✓	5	John	44	US	B
✗	6	Emma	44	US	B

Querying Numeric Fields

- Let's add some complexity to the question :

What are the messages from users named John, from the country US, whose age is over 40?

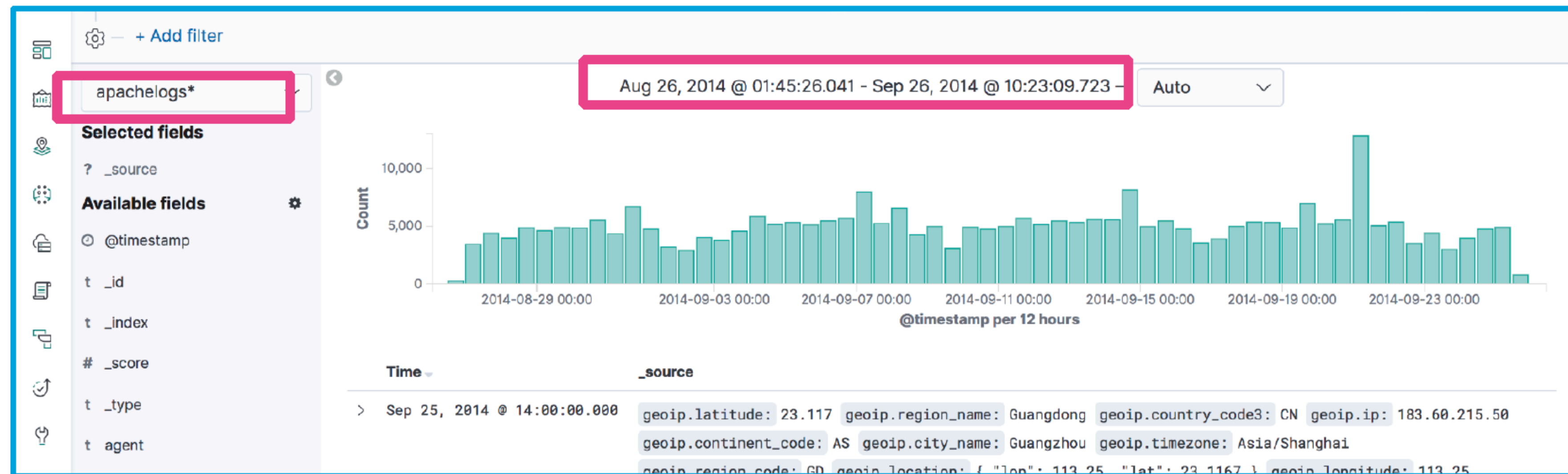
- Numbers are different that text :
 - instead of exact matches you offer have relations:
 - less that (<)
 - less than or equal (<=)
 - greater that (>)
 - greater that or equal (>=)
- Now, you can rewrite the query as :

user:john and country:us and age>40



Query "Context"

- Query includes criteria about where to search, bases on:
 - distribution in Elasticsearch (Index Pattern)
 - distribution in time range (Time Picker)
- Make sure to set the correct index pattern and time frame :

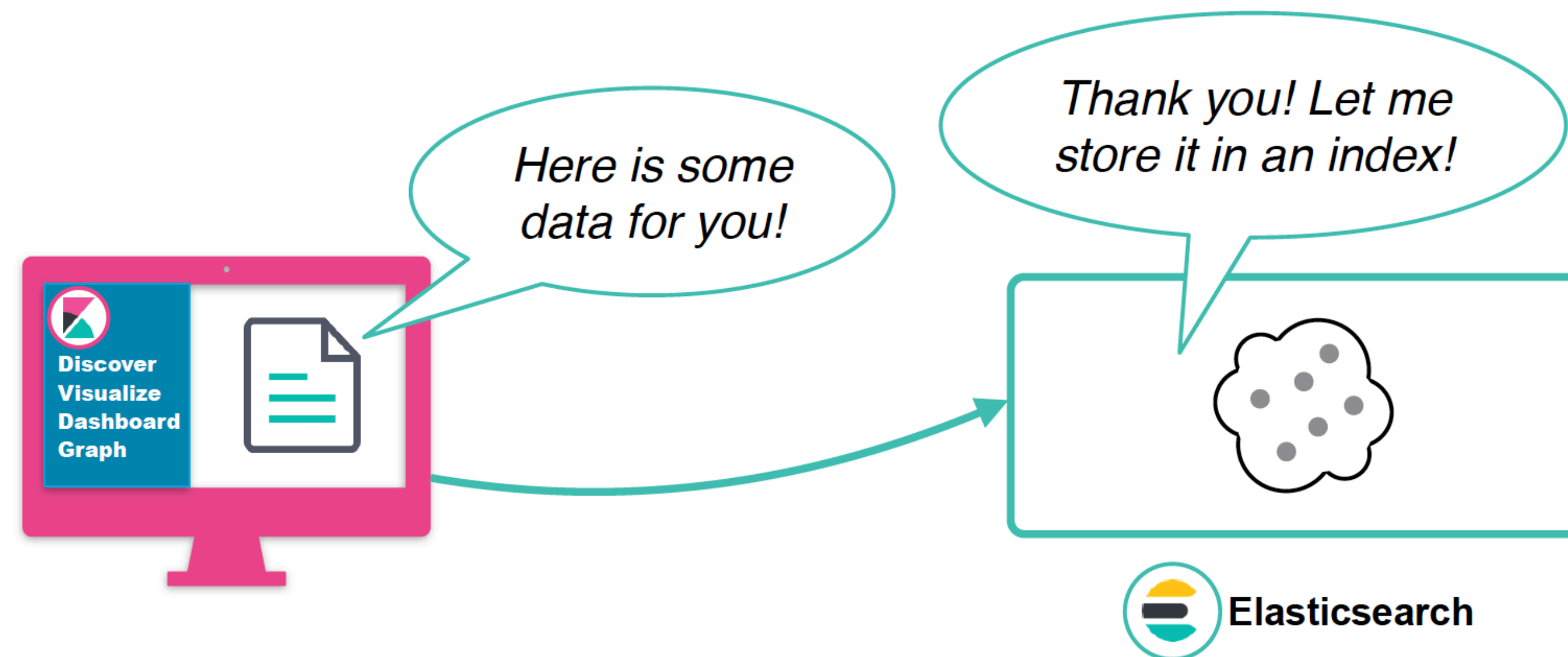


Summary

- The Discover interface allows you to explore the different aspects of your data
- The most common mistake in the Discover interface is not checking the **index pattern** and **time picker**
- The query bar can be used to search all the data inside Elasticsearch
- The document table can be customized to display a table of only selected fields

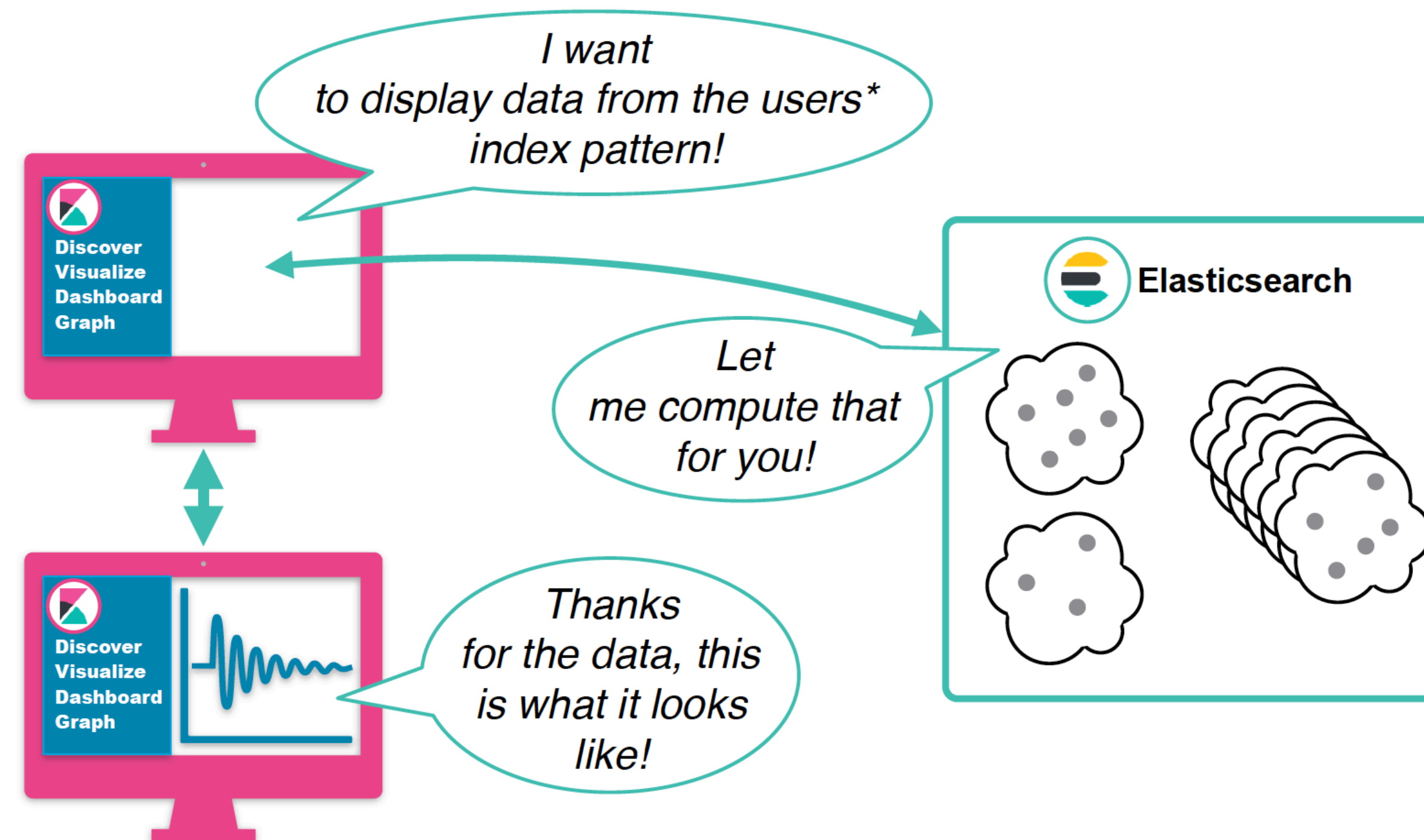
Uploading Data through Kibana

- Kibana is a powerful tool, but it does not store data
- Data is stored in Elasticsearch
- Kibana provides a tool allowing a user to import a file
 - The data will be shipped to Elasticsearch
 - if necessary, the data can be processed (transformed)



Kibana a Visualization Tool

- Kibana is a tool that can be used to visualize data
- Kibana is going to leverage Elasticsearch



Elasticsearch Powers Kibana

- Elasticsearch powers Kibana
- Does that mean that people using Kibana need to be experts in Elasticsearch ?
 - ... not really
- Kibana is a tool that anybody can use :
 - knowing the concepts Elasticsearch can help
 - ... but you can always start with ***Kibana Lens***

Kibana Lens

- Kibana Lens is an easy-to-use and intuitive UI
- It aims at simplifying the creation of visualizations :
 - offers drag and drop feature
 - exploration of the different type of visualizations
 - creation of a visualization in just a few clicks
- It is the simple way of creating visualization in Kibana
- All visualizations created using Kibana Lens can be used in a dashboard

Summary

- Elasticsearch is used to compute the data that is displayed in Kibana
- Someone does not need to be an expert in Elasticsearch to be able to use Kibana
- Kibana Lens is a type of visualization introduced in order to make the creation of visualizations easy

Module 2 : Search

- Searching on Strings
- Filters

Lesson 1 : Searching on Strings

Improving Search

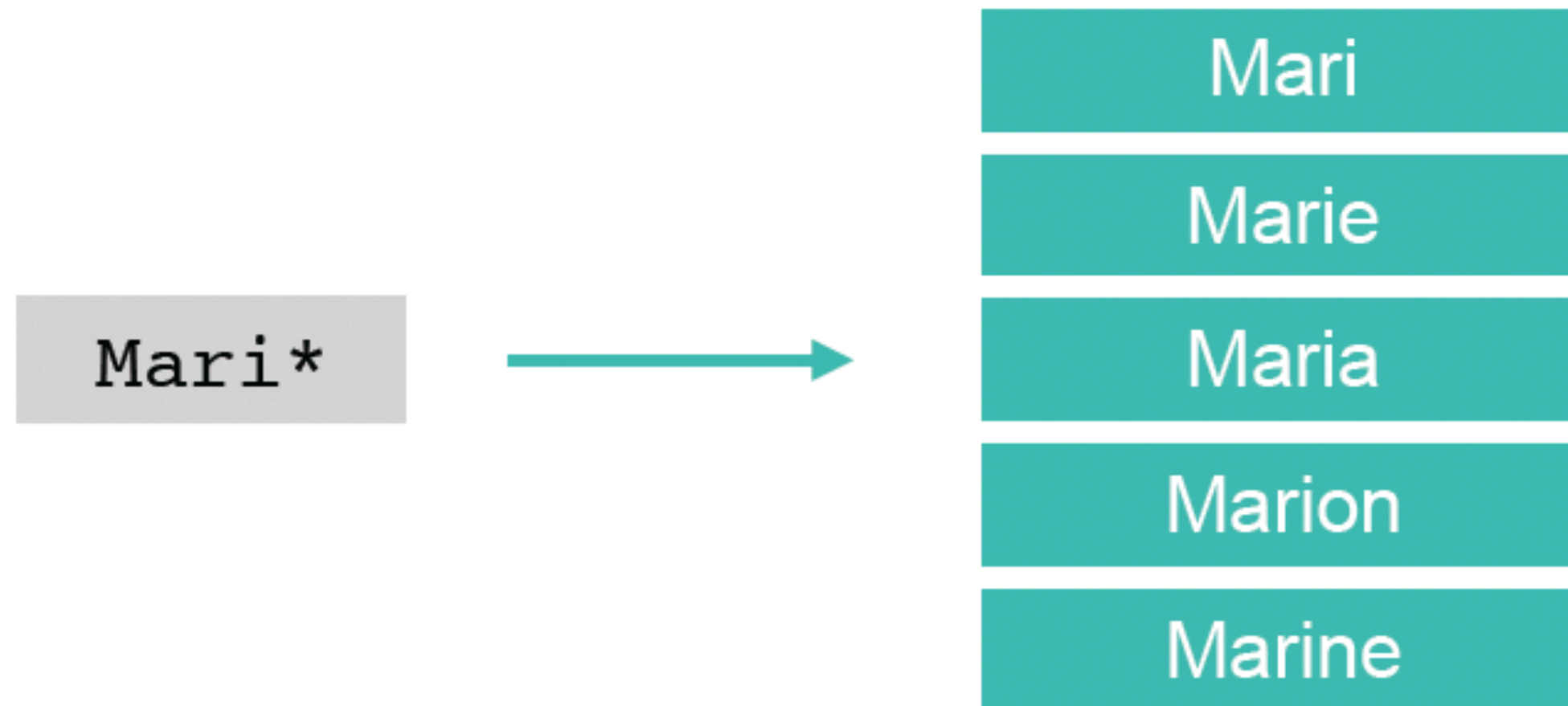
- Search is central to Elastic
 - in Kibana, ability to search is embedded in almost every page
- However, search is not limited to words we type in a search bar :
 - or filtering fo messages in a specific interval
- Searches may involve :
 - fuzzy queries
 - looking for patterns using regex or wildcard
 - searching with bias over certain fields

Wildcard Overview

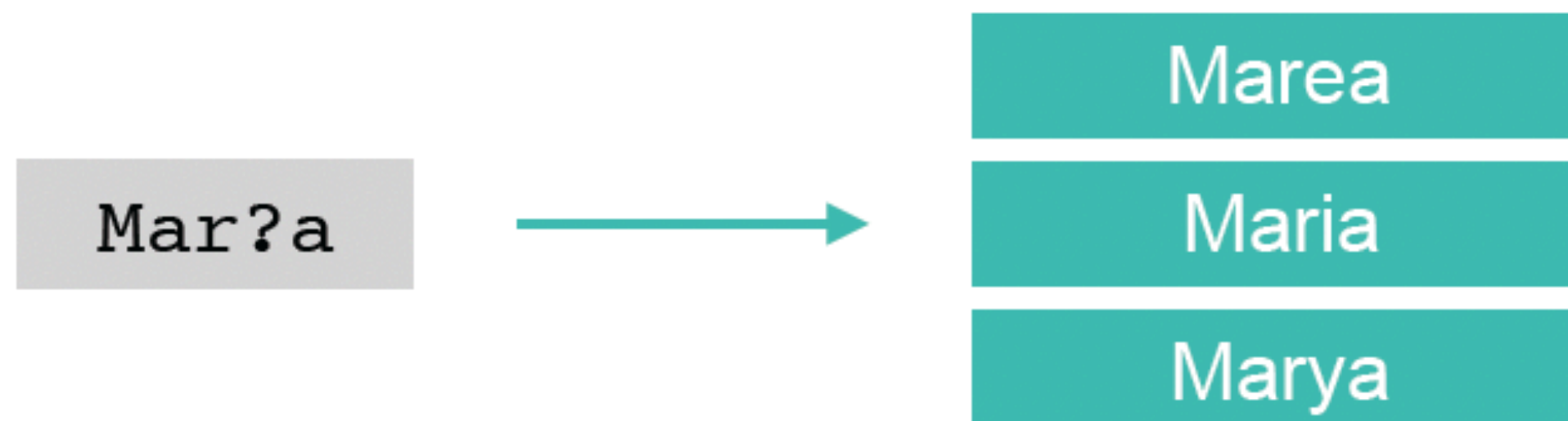
- In many cases you may want to search for a pattern:
 - `var/lib/elasticsearch/conf/elasticsearch.yml`
`var/lib/elasticsearch/conf/jvm.options`
`var/lib/elasticsearch/conf/log4j2.properties`
 - `jump`, `jumps`, `jumping`, `jumped`
- The *wildcard query* supports two operators:
 - `*` that matches any character sequence (including empty ones)
 - `?` that matches a single character (not available with KQL)

Wildcard Operators

- Both * and ? can be used to search part of a string
- Let's search for every word that start with **Mari**:



- Let's search for every word that starts with **Mar?a**
 - where ? can be any single character



Searching for IP Addresses

- Using the search bar it is also possible to search over IP addresses
- The search bar supports the **CIDR notation**
- Using the Lucene syntax you will need to add quotes to your query

Lucene

```
user.ip_address:"192.168.60.30/18"
```

KQL

```
user.ip_address:192.168.60.30/18
```



192.168.0.0

...

192.168.63.255

The Lucene Syntax

- The default syntax for the query bar is **KQL** (Kibana Query Language)
- You can change it to the **Lucene** syntax in the query bar, if needed
- The advantage of the Lucene query is that:
 - it offers more features such as regex or fuzzy queries
 - people familiar with Elasticsearch will be able to use it easily

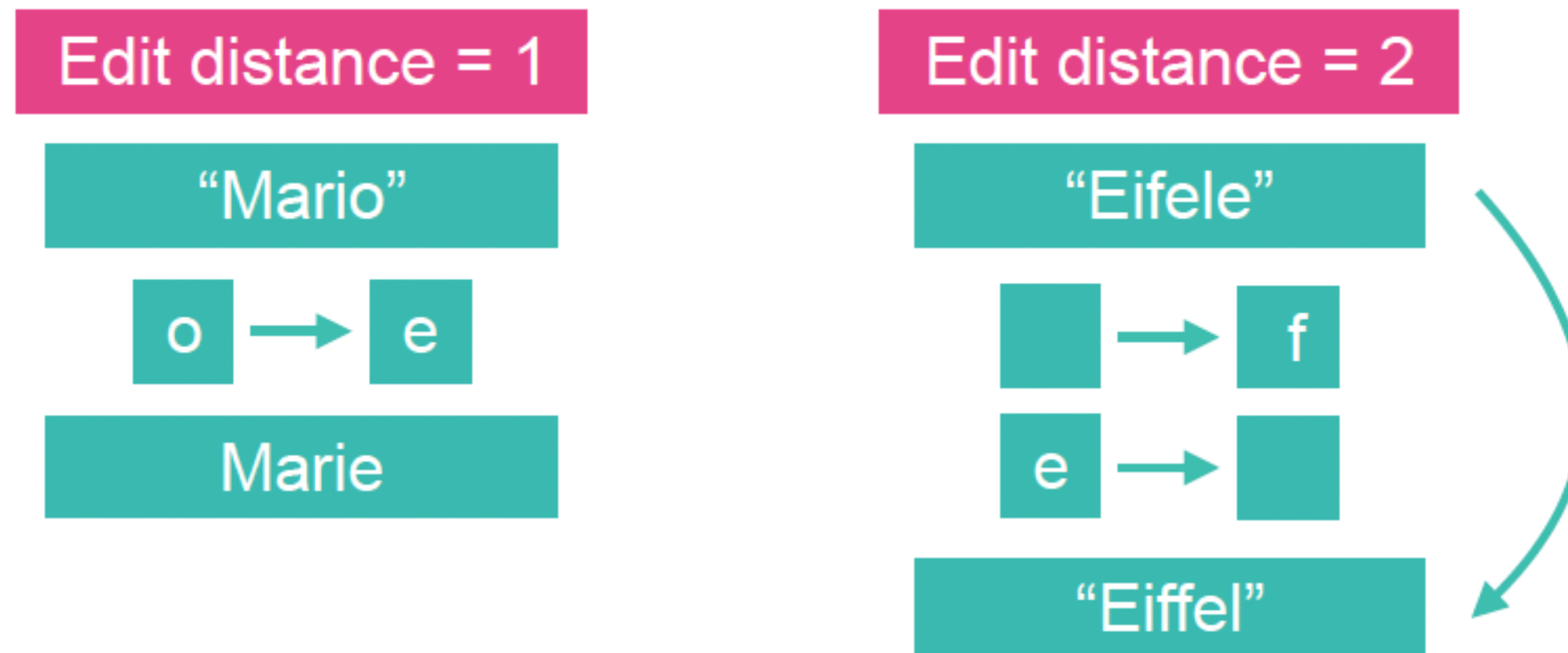
KQL 

Lucene 

- A third language called the query DSL can be used (this will be covered later in this training)

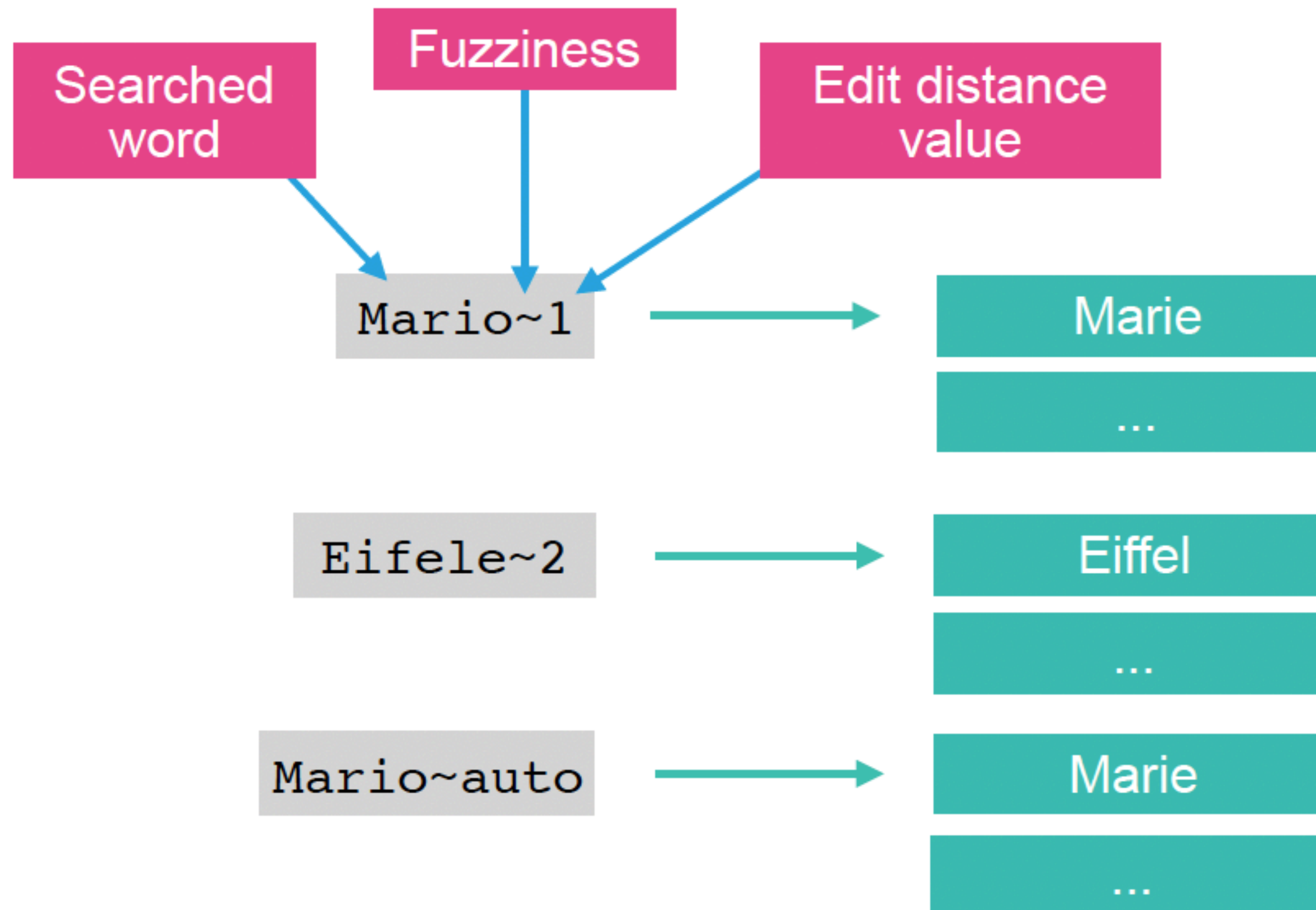
Fuzzy Search

- Users expect search applications to adapt to spelling errors
- We can use an approach known as **edit distance**:
 - edit distance is a function that counts the number of edits required to change one word into another
 - Damereau-Levenshtein distance is a popular method



Fuzzy Search

- The edit distance can be defined per query term
 - allowed values are 0 (default), 1, 2, and auto
 - **auto** will define the fuzziness based on the length of the word



Fuzzy Search

- Be aware that **fuzzy search is computationally expensive**
 - it will not be as fast as regular searches
 - and fuzzy searches can severely slow down your cluster
- Great for incident analysis
- More computationally expensive than regular search

Regex Query

- The **regex** query greatly enhances our ability to search for patterns, using regular expressions
 - syntax is based on the [Lucene regular expression engine](#)
- For example, you can use it to find all the documents that contain a phone number in France:
 - starts with +33 followed by 9 numbers

```
/.*+33[0-9]{9}.*/
```



my phone number: +33235151151

Here: +33235151151 call me later

...

Searching for Patterns is Expensive

- Both wildcard queries and regexp queries are computationally expensive
 - they will not be as fast as regular searches
 - and they can severely slow down your cluster
- Great for incident analysis
- Not so great for common queries
 - **always avoid leading patterns** (eg: *aria)

Boosting

- When searching on multiple fields, finding something in certain fields may mean more to you than others:
 - for example, an email title is probably more relevant than the body
- In the query bar, it is possible to reflect this by **boosting** specific fields:
 - which impacts the order of the returned documents

```
subjects:instafood OR first_name:john^2
```

Searching on the **subjects**
and **first_name** fields

Boosting the score by 2
on the field **first_name**

Index Patterns

- In Kibana, Index pattern provides a summary of fields in elasticsearch index
 - along with properties that indicate whether they are suitable for
 - certain visualizations
 - exact match queries
 - full text search

	EXACT MATCH	FULL TEXT SEARCH
AGGREGATABLE	✓	
SEARCHABLE	✓	✓

Lesson 2 : Filters

Query Bar Limitations

- Imagine you have multiple search criteria:
 - `age>30`
 - `subjects:art`
 - `subjects:fashion`
 - `subjects:summer`
 - `first_name:Claire`
- Often times, you may want the ability to turn on/off these criteria individually
- Using the query bar will require a lot of typing and deletions to accomplish this
- Filters can come in very handy to do just that...

Filters Overview

- A filter is a graphical control equivalent to a search criterion
- Once defined a filter can be:
 - enabled/disabled
 - pinned
 - negated/positivized
 - dropped
 - edited

Filters Customization

- Internally filters are transformed to a query that uses the query DSL syntax
- There are two ways of customizing a filter:
 - adding a label to the filter to quickly identify it
 - redefine the way the filter behaves by editing the query DSL (this is going to be covered later)

Filter in Discover

- Filters can be used to quickly filter and explore data:
 1. create multiple relevant filters
 2. type a query
 3. disable/enable filter to explore data

Filter Across Kibana

- Filters can be used to navigate across multiple Kibana interfaces
 1. create filters in discover
 2. pin interesting filters
 3. navigate to Visualize
 4. create a visualization

Controls Visualization

- Some visualizations are designed to create filters
 - the **controls visualization** provides an intuitive way to let analysts filter for specific fields
- With the **controls** visualization, you can:
 - add a slider for numeric values
 - add an options list to pick keywords

Filter and Query Bar

- Filters and the query bar complement each other
- It is possible to type multiple search criteria in the query bar
 - however when executed all criteria will be applied at the same time
- Filters provide the ability to quickly enable/disable criteria individually, using mouse clicks
- In the background, filters gets translated to queries and they will have similar performance

Module 2 : Visualization

- Behind the visualizations
- More types of visualizations
- Improving visualizations

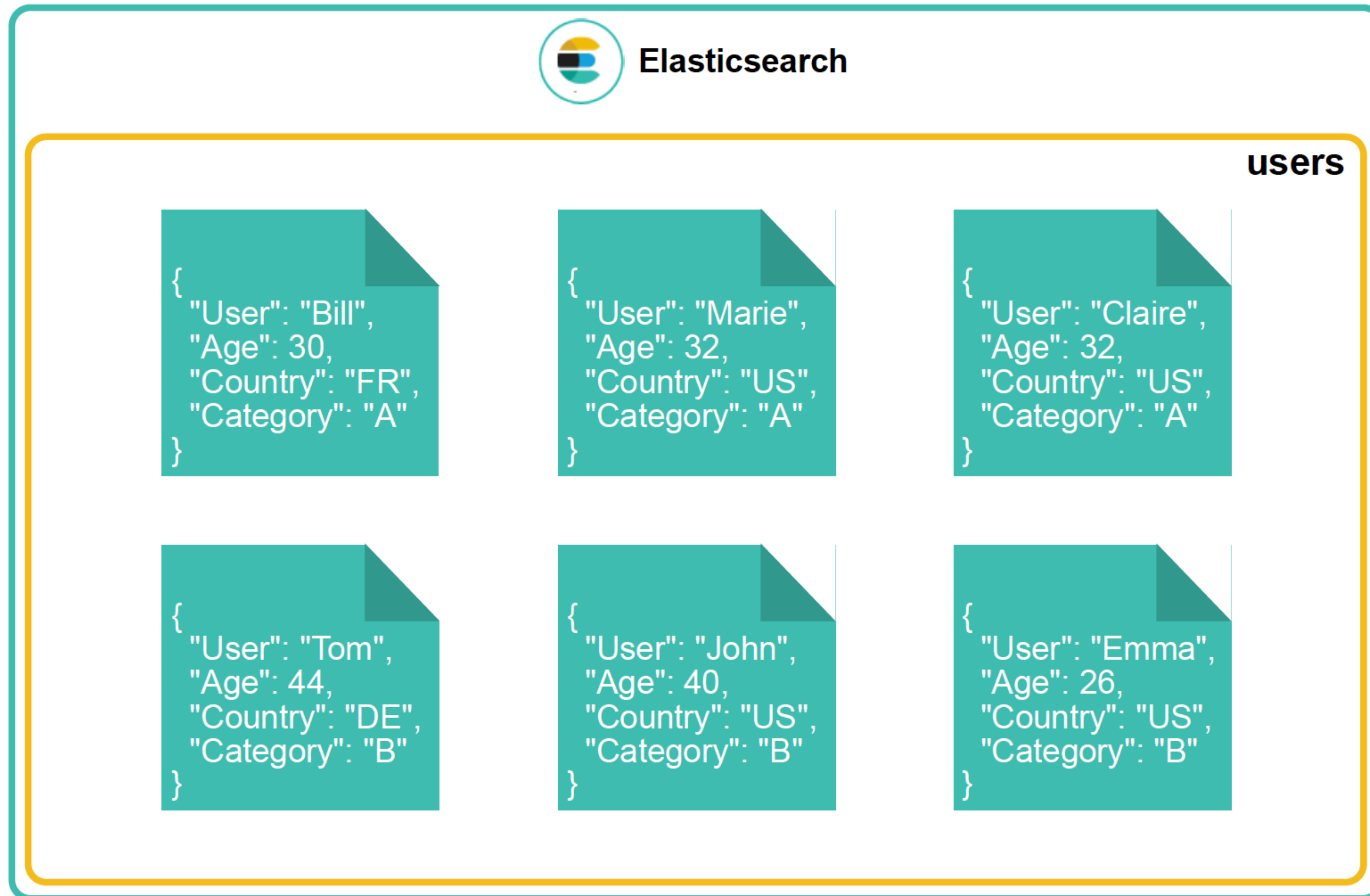
Overview

- Data is often complex and involves many dimensions
- Often, we want summarized insights:
 - slices based on specific attributes
 - calculations based on specific attributes
 - ...
- Spreadsheets might fulfill this using a "**pivot table**"
- In the Elastic Stack we call the equivalent functionality an **aggregation**
- All aggregations are performed in Elasticsearch
 - Kibana just displays the results

A Simple Example: Spreadsheet

<i>id</i>	<i>user</i>	<i>age</i>	<i>country</i>	<i>category</i>
1	Bill	30	FR	A
2	Marie	32	US	A
3	Claire	32	US	A
4	Tom	44	DE	B
5	John	40	US	B
6	Emma	26	US	B

A Simple Example: Elasticsearch



Metrics Aggregations

- Calculate numerical values over a set of documents
- similar to how **values** are **summarized** in a **pivot table** for a specific column
- mathematical operation that output :
 - a single value (eg., **avg**, **sum**, **min**, **max**, **unique count**)
 - or multiple values (eg., **percentiles**, **percentile_ranks**)

A Simple Average Using Pivot Table

<i>id</i>	<i>user</i>	<i>age</i>	<i>country</i>	<i>category</i>
1	Bill	30	FR	A
2	Marie	32	US	A
3	Claire	32	US	A
4	Tom	44	DE	B
5	John	40	US	B
6	Emma	26	US	B

Pivot table definition

<i>Rows</i>	<i>Values</i>
	AVG of age



Pivot table

AVG of age
34

A Simple Average Using Aggregations



Elasticsearch

```
{  
  "User": "Bill",  
  "Age": 30,  
  "Country": "FR",  
  "Category": "A"  
}
```

```
{  
  "User": "Marie",  
  "Age": 32,  
  "Country": "US",  
  "Category": "A"  
}
```

```
{  
  "User": "Claire",  
  "Age": 32,  
  "Country": "US",  
  "Category": "A"  
}
```

```
{  
  "User": "Tom",  
  "Age": 44,  
  "Country": "DE",  
  "Category": "B"  
}
```

```
{  
  "User": "John",  
  "Age": 40,  
  "Country": "US",  
  "Category": "B"  
}
```

```
{  
  "User": "Emma",  
  "Age": 26,  
  "Country": "US",  
  "Category": "B"  
}
```

```
{  
  "aggregations": {  
    "avg_of_age": {  
      "avg": {  
        "field": "age"  
      }  
    }  
  }  
}
```



```
"aggregations" : {  
  "avg_of_age" : {  
    "value" : 34.0  
  }  
}
```

Bucket Aggregations

- Buckets aggregations allow you to :
 - **slice** the data
 - similar to grouping by values in **rows** or **columns** in a **pivot table**
 - create ***buckets***
 - collection of documents that share a common criterion
 - can have one or more metrics associated with it
 - number of documents (doc count) per bucket is the default metric

Simple Bucket Using a Pivot Table

<i>id</i>	<i>user</i>	<i>age</i>	<i>country</i>	<i>category</i>
1	Bill	30	FR	A
2	Marie	32	US	A
3	Claire	32	US	A
4	Tom	44	DE	B
5	John	40	US	B
6	Emma	26	US	B

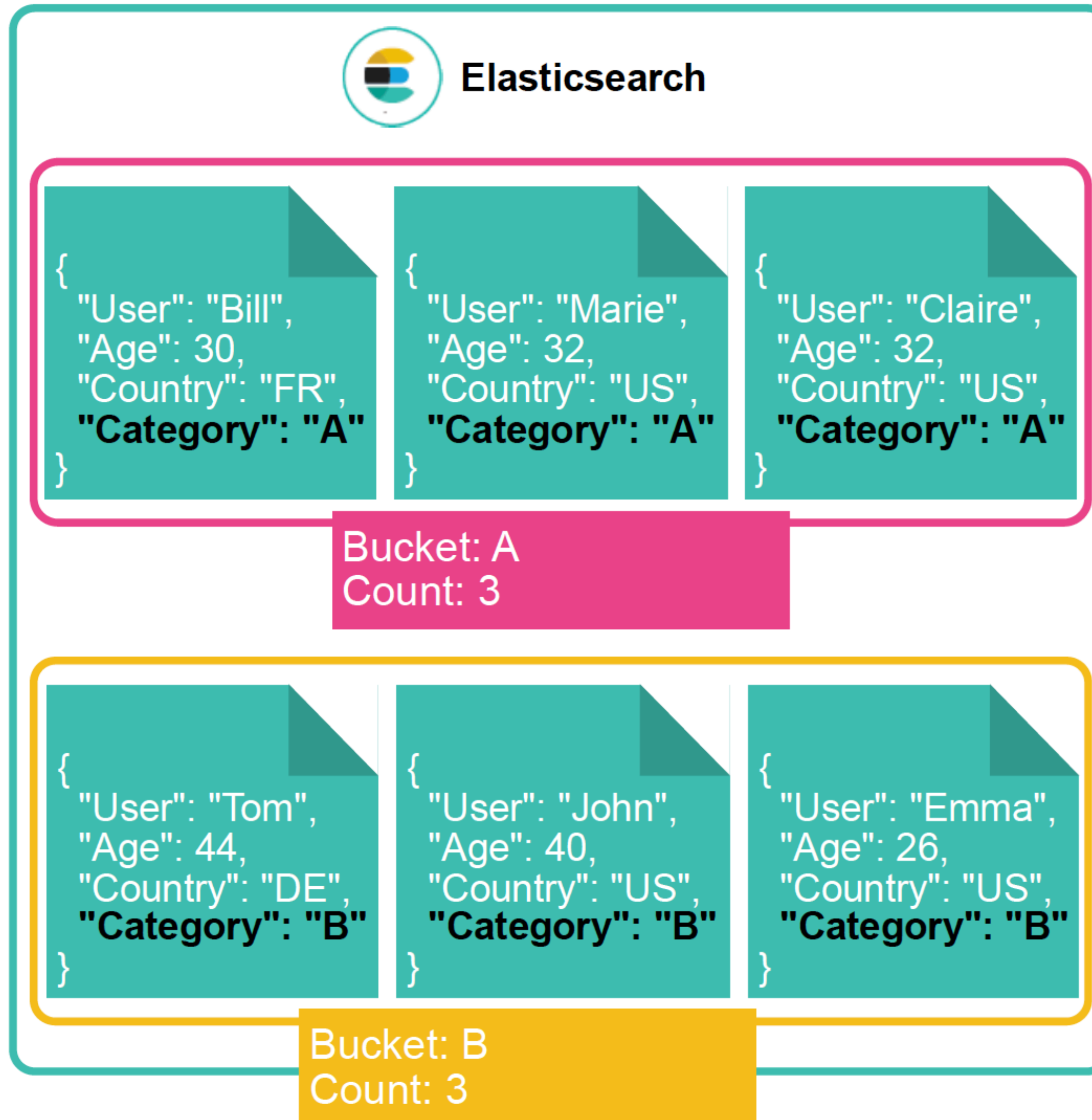
Pivot table definition

<i>Rows</i>	<i>Values</i>
Order ASC by category	COUNT of id

Pivot table

<i>category</i>	<i>COUNT of id</i>
A	3
B	3

Simple Bucket Aggregation



```
{
  "aggregations": {
    "categories": {
      "terms": {
        "field": "category"
      }
    }
  }
}
```

↓

```
"aggregations": {
  "categories": {
    "buckets": [
      {
        "key": "A",
        "doc_count": 3
      },
      {
        "key": "B",
        "doc_count": 3
      }
    ]
  }
}
```

Adding Metrics

<i>id</i>	<i>user</i>	<i>age</i>	<i>country</i>	<i>category</i>
1	Bill	30	FR	A
2	Marie	32	US	A
3	Claire	32	US	A
4	Tom	44	DE	B
5	John	40	US	B
6	Emma	26	US	B

<i>Rows</i>	<i>Values</i>	<i>category</i>	<i>COUNT of age</i>	<i>AVG of age</i>
<i>Order ASC by category</i>	<i>COUNT of age</i>	A	3	31.33
	<i>AVG of age</i>	B	3	36.66

Adding Metrics



Elasticsearch

```
{
  "User": "Bill",
  "Age": 30,
  "Country": "FR",
  "Category": "A"
}
```

```
{
  "User": "Marie",
  "Age": 32,
  "Country": "US",
  "Category": "A"
}
```

```
{
  "User": "Claire",
  "Age": 32,
  "Country": "US",
  "Category": "A"
}
```

Bucket: A
Count: 3
Avg of age: 31.33

```
{
  "User": "Tom",
  "Age": 44,
  "Country": "DE",
  "Category": "B"
}
```

```
{
  "User": "John",
  "Age": 40,
  "Country": "US",
  "Category": "B"
}
```

```
{
  "User": "Emma",
  "Age": 26,
  "Country": "US",
  "Category": "B"
}
```

Bucket: B
Count: 3
Avg of age: 36.66

```
"aggregations": {
  "categories": {
    "terms": {
      "field": "category"
    },
    "aggregations": {
      "avg_age_per_category": {
        "avg": {
          "field": "age"
        }
      }
    }
  }
}
```



```
"aggregations": {
  "categories": {
    "buckets": [
      {
        "key": "A",
        "doc_count": 3,
        "avg_age_per_category": {
          "value": 31.33
        }
      },
      {
        "key": "B",
        "doc_count": 3,
        "avg_age_per_category": {
          "value": 36.66
        }
      }
    ]
  }
}
```


Nesting Rows/Columns in a Pivot Table

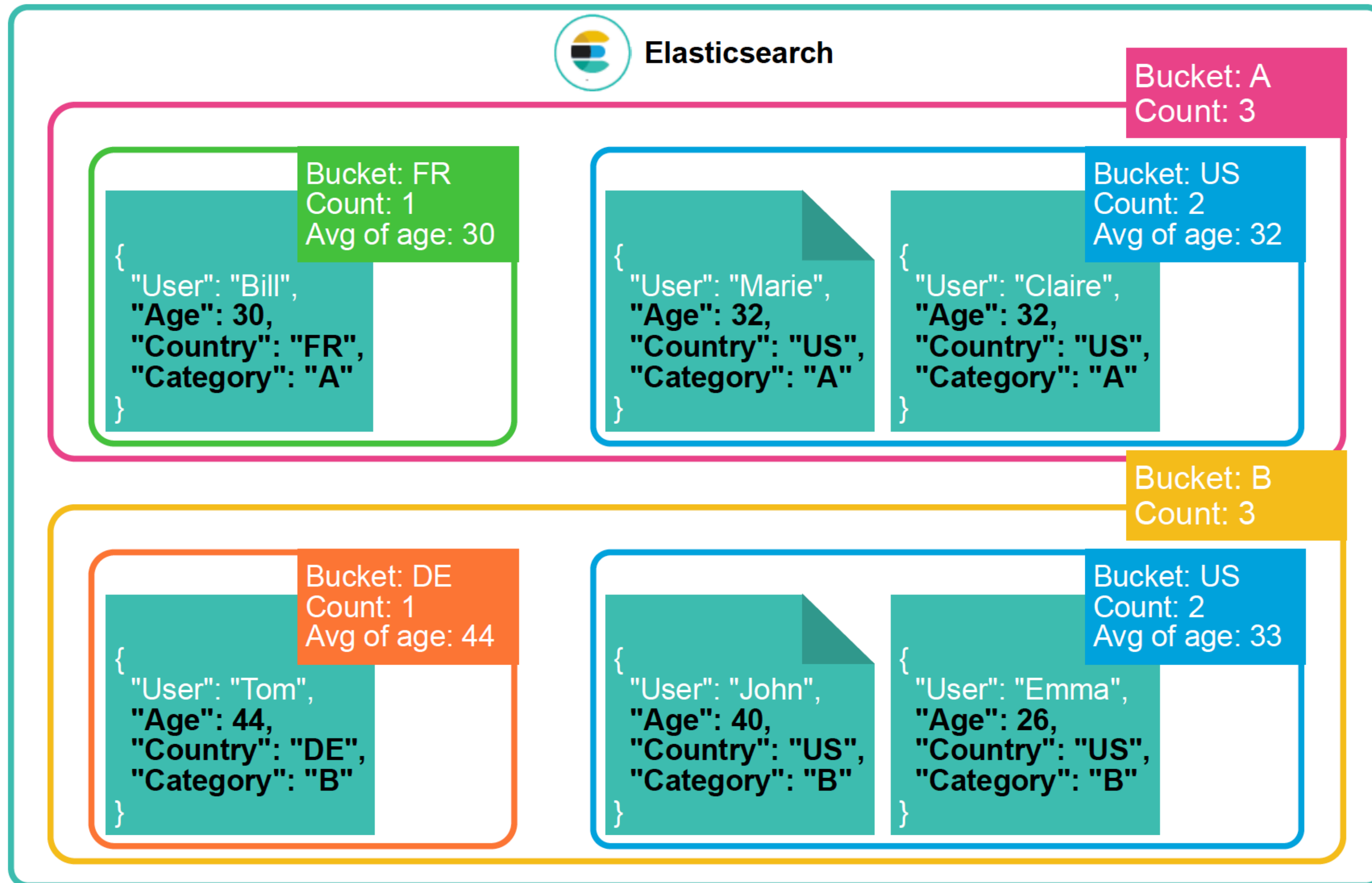
<i>id</i>	<i>user</i>	<i>age</i>	<i>country</i>	<i>category</i>
1	Bill	30	FR	A
2	Marie	32	US	A
3	Claire	32	US	A
4	Tom	44	DE	B
5	John	40	US	B
6	Emma	26	US	B

<i>Rows</i>	<i>Values</i>
Order ASC by category	COUNT of age
Order ASC by country	AVG of age



<i>category</i>	<i>country</i>	<i>COUNT of age</i>	<i>AVG of age</i>
A	FR	1	30
	US	2	32
B	DE	1	44
	US	2	33

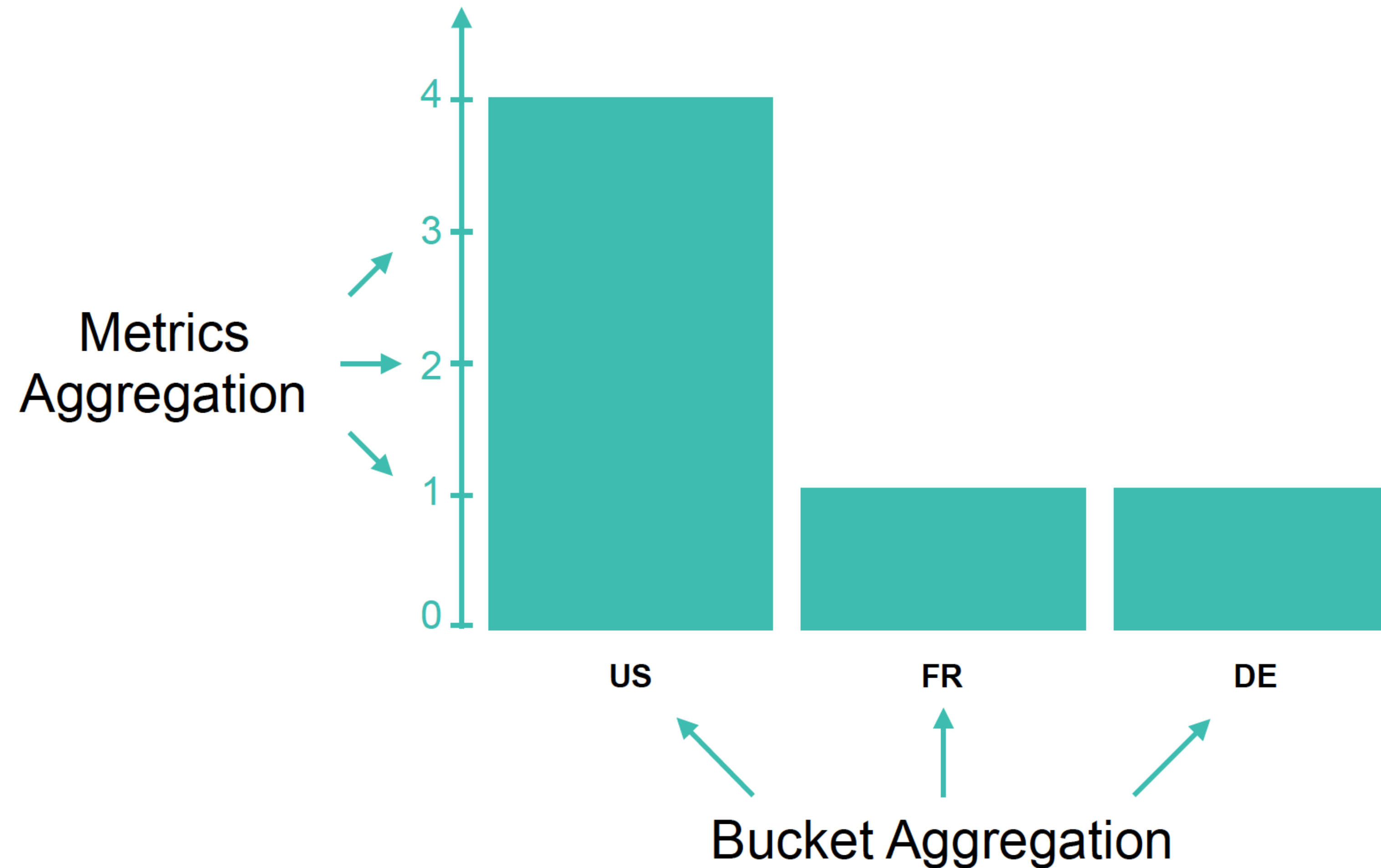
Adding Sub-Bucket Aggregation



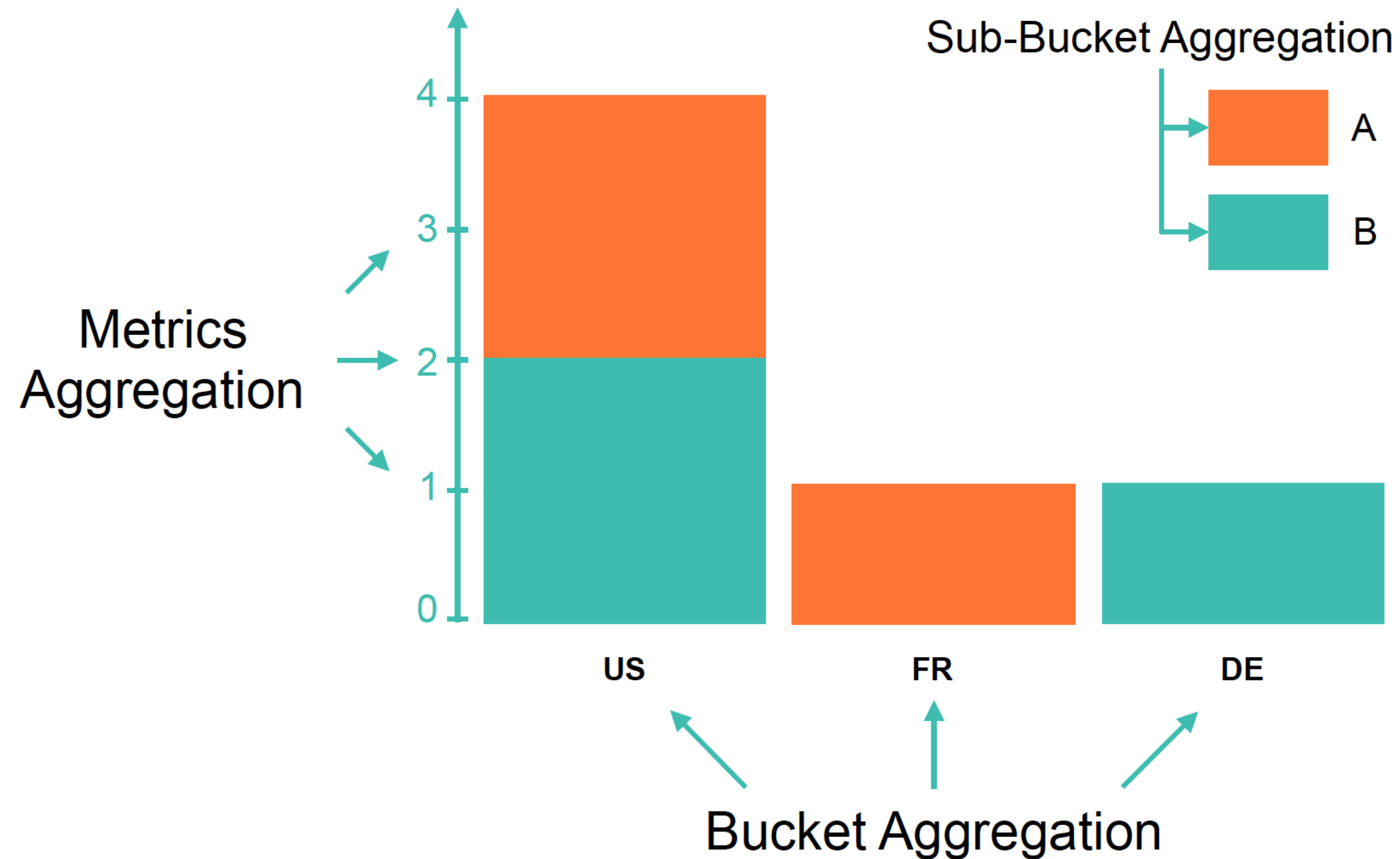
Metrics Aggregations



Bucket Aggregations



Sub-bucket Aggregations



Summary

- Kibana renders visualizations using the results of Elasticsearch aggregations
- There are two main types of aggregations:
 - metric
 - bucket
- Metric aggregations are used to compute numeric values
- Bucket aggregations are used to group data together

Pivot Table in Kibana

- Aggregations and pivot tables are two concepts that are very similar
- By using the **data table** visualization it is possible to have a "pivot table" like structure in Kibana
- What aggregations are needed in order to generate the following table ?

<i>Category</i>	<i>Country</i>	<i>Count</i>	<i>Average(age)</i>
Category A	US	2	32
Category A	FR	1	30
Category B	US	2	33
Category B	DE	1	44

Bucket Aggregations

- We need the following to create the table:
 - a **bucket aggregation** of type **terms** on the category field first
 - a **sub-bucket aggregation** of type **terms** on the country field
 - a **metric aggregation** of type **count** (default metric aggregation)
 - another **metrics aggregations** of type **average** on the **age field** on all the generated buckets

Table Visualization

- By default data table in Kibana will have only one row
 - Showing a count of all documents

<i>Count</i>
6

- The row can be split by first applying a terms aggregation on the category field

<i>Category</i>	<i>Count</i>
Category A	3
Category B	3

Table Visualization

- Resulting rows can then be split again by applying another terms aggregation on the country field

Category	Country	Count
Category A	US	2
Category A	FR	1
Category B	US	2
Category B	DE	1

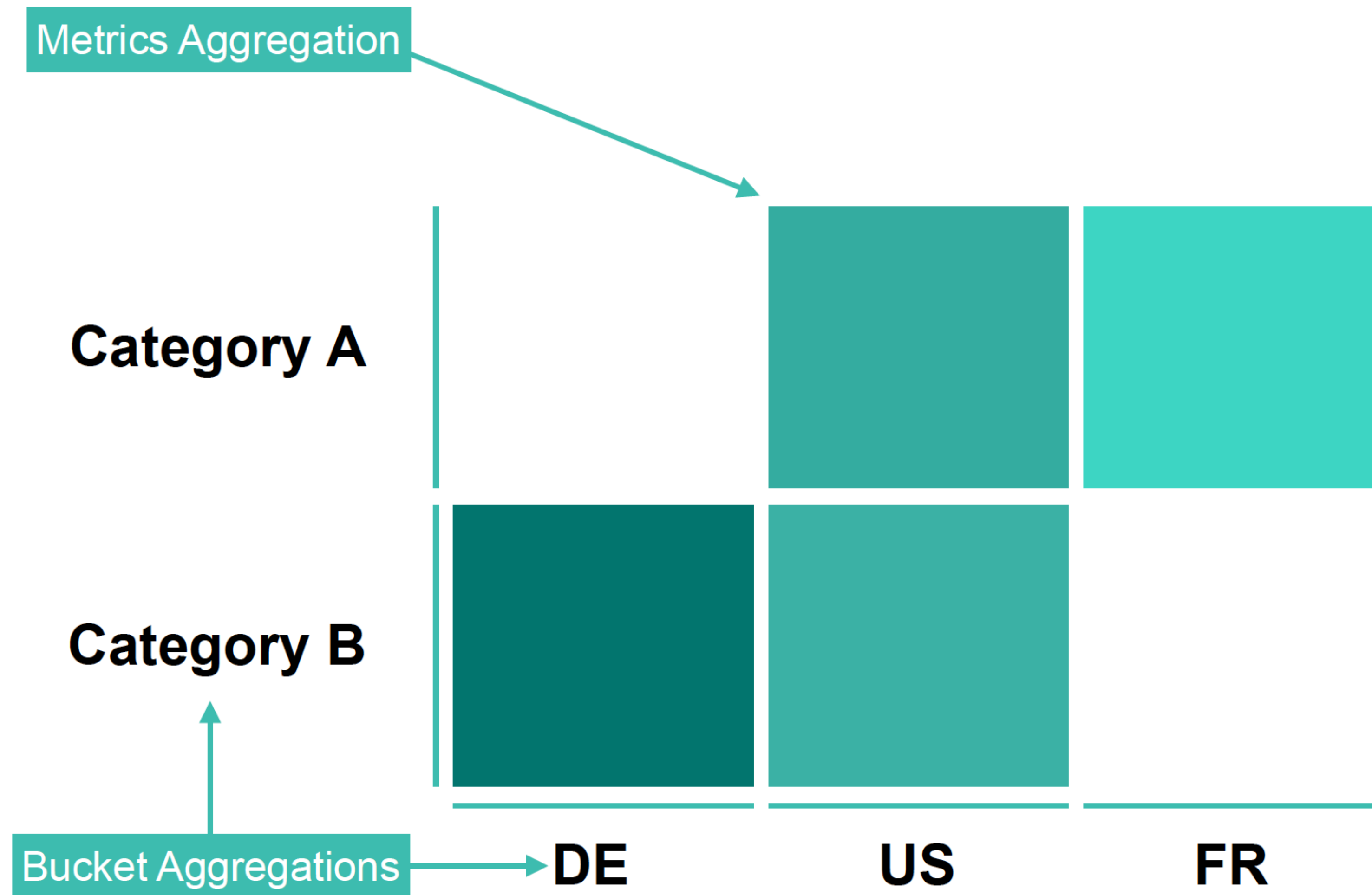
- A column can then be added to the table by adding the average metrics aggregation on the age field :

Category	Country	Count	Average(age)
Category A	US	2	32
Category A	FR	1	30
Category B	US	2	33
Category B	DE	1	44
Total		6	139

Customizing Data Table

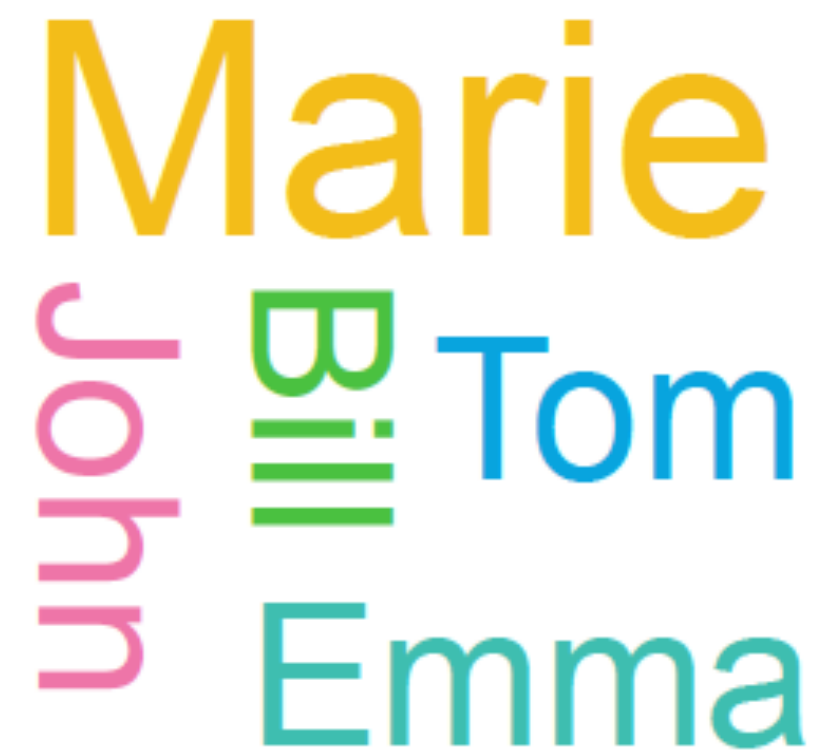
- Every row in a table is the result of a **bucket aggregation**
- The number of rows displayed in the table can be customized using the option tab
- Tables can span multiple pages and you can paginate to subsequent pages using the link at the bottom of the table
- It is possible to apply the metric aggregations at the parent level using the option tab
- A grand total of all the metrics can also be computed

Heat Map Overview



Tag Cloud Overview

- Tag cloud is way to visually represent words in a dataset
- It supports only two bucket aggregations:
 - *terms aggregation*
 - *significant term aggregation*
- Every word is the result of the bucket aggregation
- The size of a word is a function of the result of the metric aggregation (default to count)



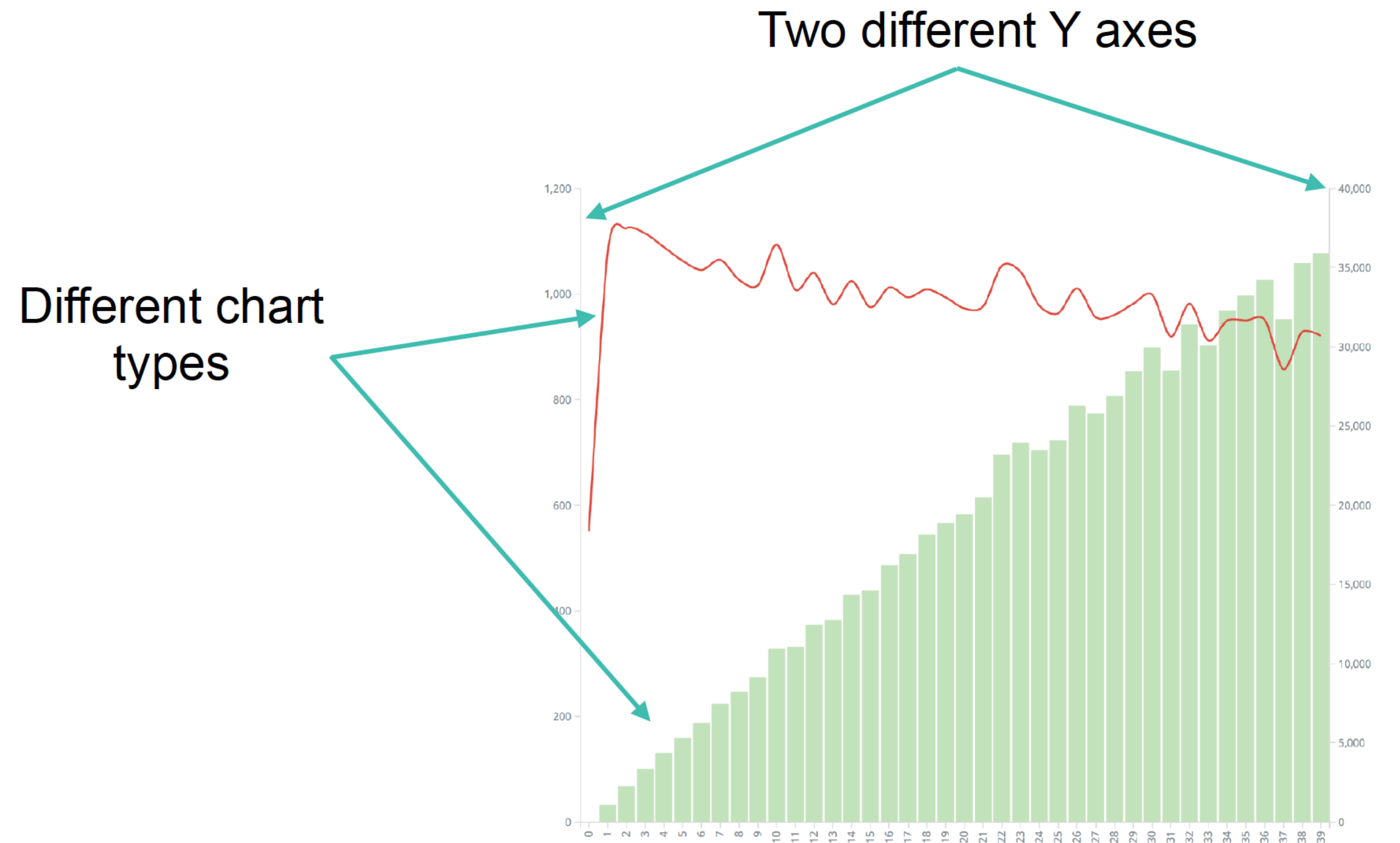
Marie
John Bill Tom
Emma

Visualizing Multiple Metrics

- Multiple metrics can be displayed side by side in the same visualization
- Having multiple metrics can make the visualization hard to read for the following reasons :
 - the metrics have different scale
 - the metrics are stacked (or not, depending of the purpose of the visualization)
 - the metrics have same representations when comparing different variables (bars, lines, area)
 - poor color choice

Visualizing Multiple Metrics

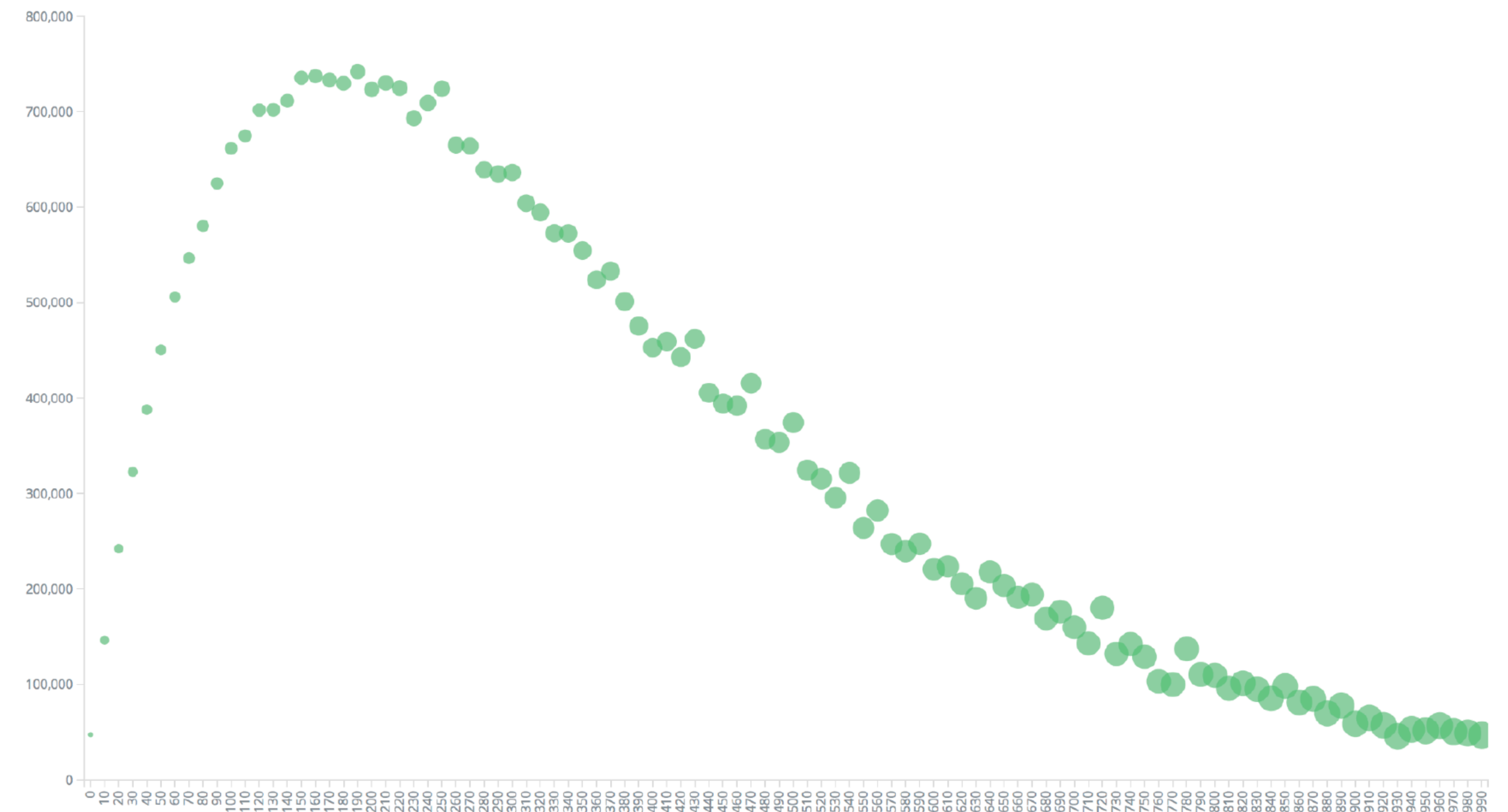
- To overcome these challenges, every metrics aggregation can be customized with its own style:
 - color
 - axis
 - properties
- Each metric can be displayed differently :
 - lines
 - bars
 - areas



Bubble Chart

- Bubble chart can be built by defining a new metric:

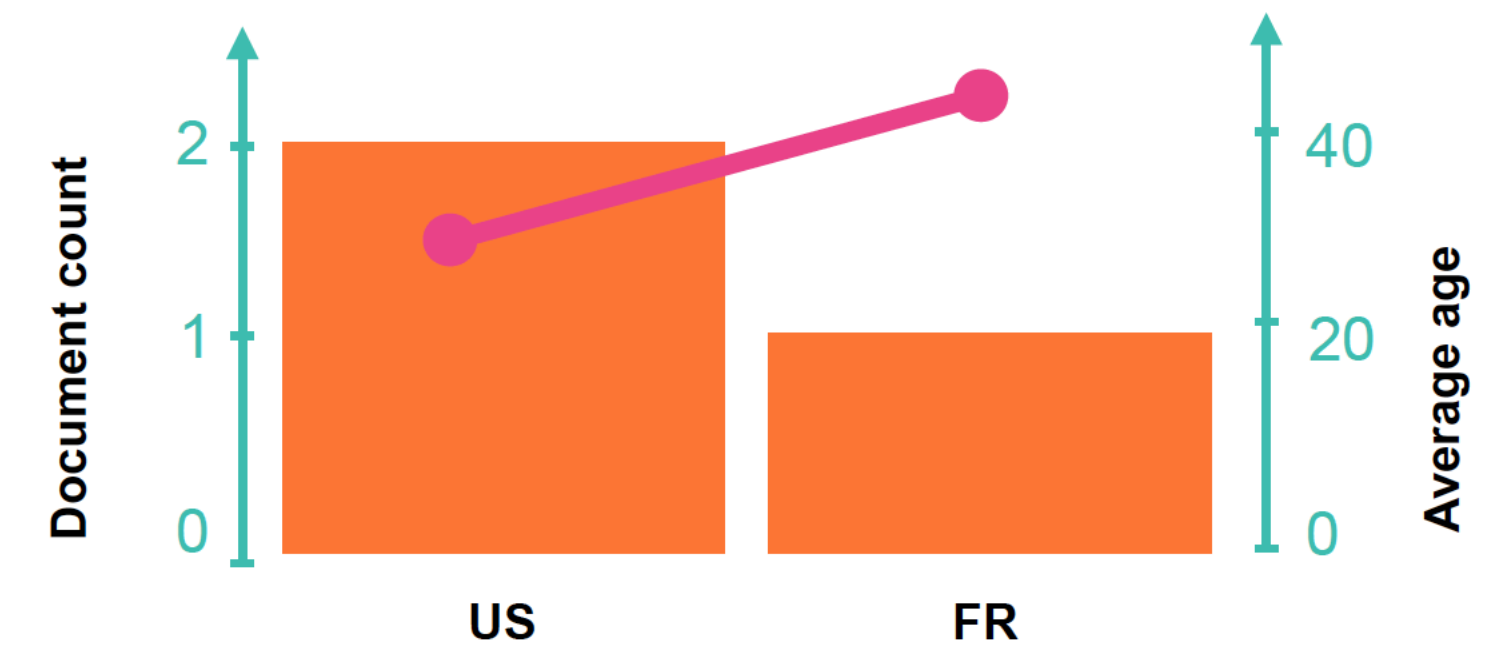
1. define X-axis
2. define Y-axis
3. define dot size
4. change visualization to display a line
5. uncheck "show line"



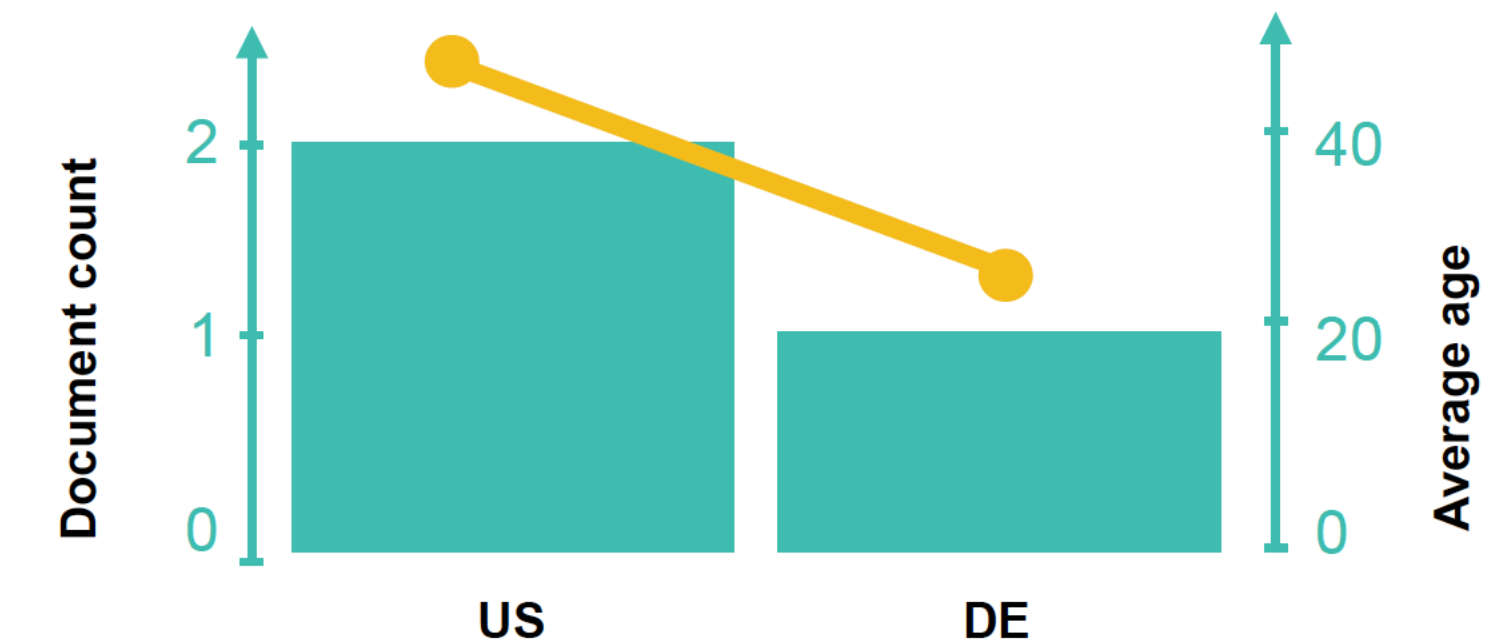
Visualizing Multiple Metrics

- Sometimes, multiple buckets or sub-buckets on a single visualization may not be suitable :
 - buckets or sub-buckets may not be related
 - too many metrics on the same visualization can make it busy
- In Kibana it is possible to display multiple charts in the same visualization by using split chart

Category A



Category B

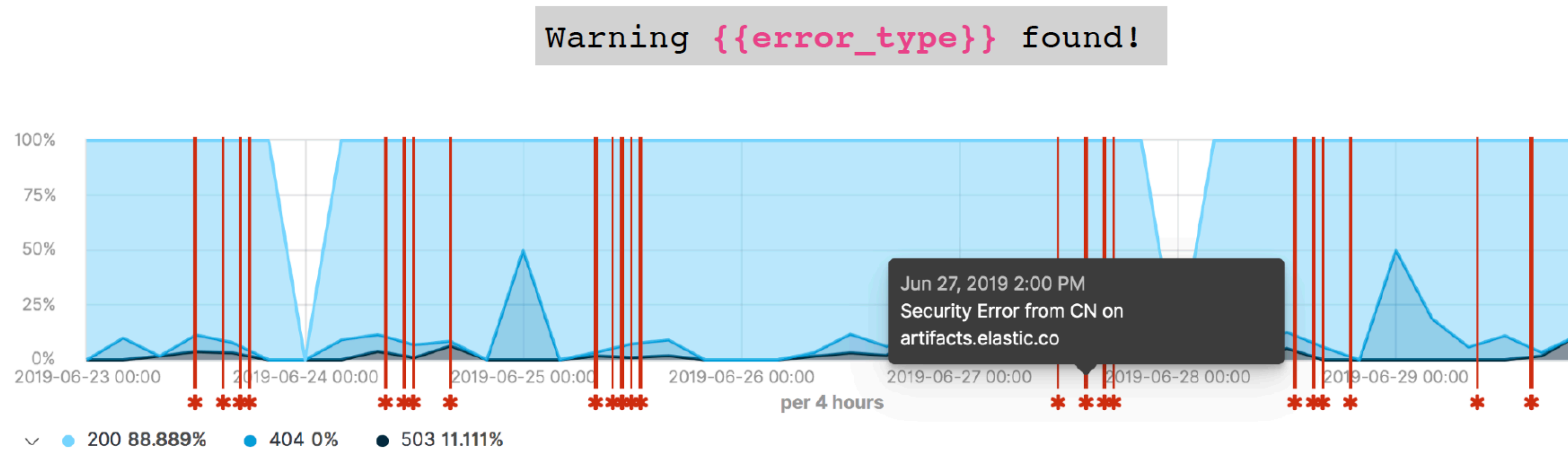


Time Series Visual Builder (TSVB)

- combines a wide range of aggregations to analyze time series data in a meaningful way
 - math (to apply mathematic operations to the data)
 - static values
 - overall maximum and minimum
 - and many more
- supports many customization options in how data is visualized in chart
 - background colors
 - axis
 - ...
- supports combining multiple index patterns in the same visualization
- supports annotations

Annotations

- Using TSVB, it is possible to add **annotations** to highlight important events in your time series
- Every annotation comes with a tooltip that can be customized using a templating language called **Mustache**
- *Mustache* has a fairly straightforward syntax:



Timelion

- Timelion is another powerful time series data visualizer in Kibana
 - geared towards users who prefer writing code
 - driven by a simple expression language
 - uses "." chained functions
- query bar supports code completion

Markdown Visualization

- Kibana supports a Markdown tool
 - helpful when you want to include important information, instructions, and images on your dashboard.
 - add links to other dashboards (that may be related)
- In Kibana, there are two options to create a visualization that will contain Markdown
 - the Markdown tool
 - TSVB
- The main difference between the two is that in TSVB you will be able to include dynamic data that is present in the Elasticsearch cluster

Other Visualization

- TSVB is not limited to visualizing time series
- It can also be used for
 - metrics
 - gauges
 - top N
 - markdown
 - tables
- Those visualizations allow the addition of conditionals to change the color of the visualization based on a certain criterion
- You have the choice between working with the whole time series or only with the last bucket

Metric

- It is possible to visualize up to two metrics using the TSVB ***Metric visualization***
- Multiple visualizations can be generated if the documents are split using for instance a term aggregation
- Colors can be defined based on thresholds :
 - for the metric itself
 - for the background color

Top N

- The TSVB ***Top N visualization*** represents data as a vertical bar chart
- The bars can be generated using :
 - the terms aggregation
 - by defining multiple series
- Conditions can be defined to change the color of the background dynamically based on thresholds
- Color of the background can be customized

Table

- The table visualization that TSVB provides is fairly similar to the prebuild one
- It provides a way of customizing the color of the metric displayed based on thresholds at the aggregation level (column level)
- There are more aggregations available than using the prebuilt visualization

Elastic Maps Service

- The ***Elastic Maps service*** powers all the geovisualizations in Kibana including the *Maps App*
- It provides basemap tiles, shape files and other key feature to visualize **geodata**
- Developed and maintained by Elastic, it works seamlessly with the Elastic Stack
- Elastic Maps Service release are not tied to Elastic Stack releases, meaning that updates can roll in without having to update anything.

Backed by Open Source

- Elastic Maps Service produces base map tiles based on OpenStreetMap
- OpenStreetMap is licensed under the Open Data Commons Licence (ODbL)
- GeoJSON files are derived from :
 - OpenStreetMap
 - Natural Earth
 - Wikidata
- Elastic Maps Service provides base map layers for the Open Source and for the Basic, Gold, Platinum, Enterprise License

The Maps App

- The Kibana Maps app has replaced :
 - the region map visualization
 - the coordinate map visualization
 - ... but they can still be enable if needed
- The Maps app is only available with the Basic license and above

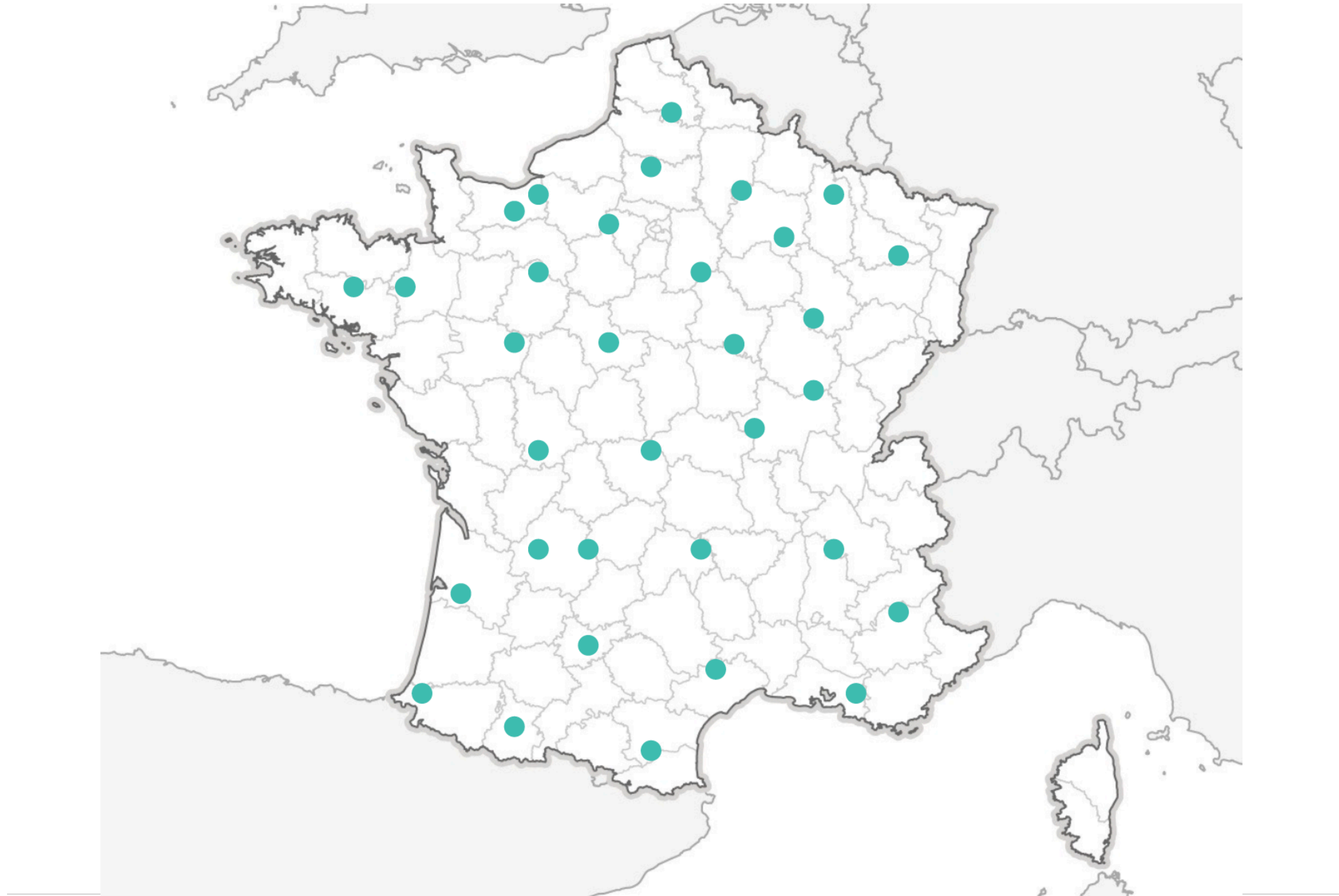
Geovisualizations

- Maps can be used to represent different entities :
 - points on a map
 - area on a map
 - path between two points
- The Maps App supports all of the above
- But to display them you need to make sure to have respectively:
 - a geo point (coordinates)
 - a keyword describing a specific area
 - two geo points

Displaying Documents

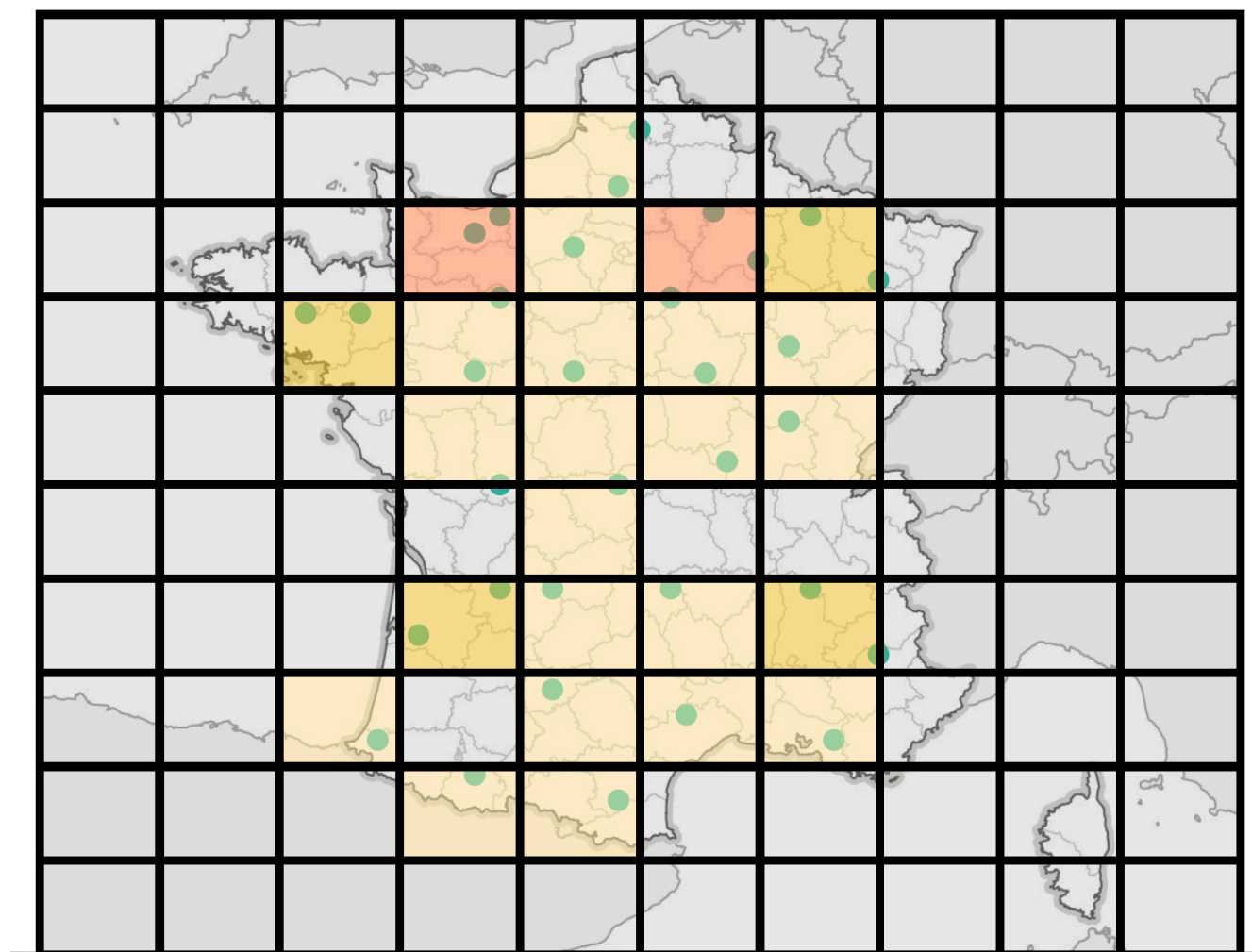
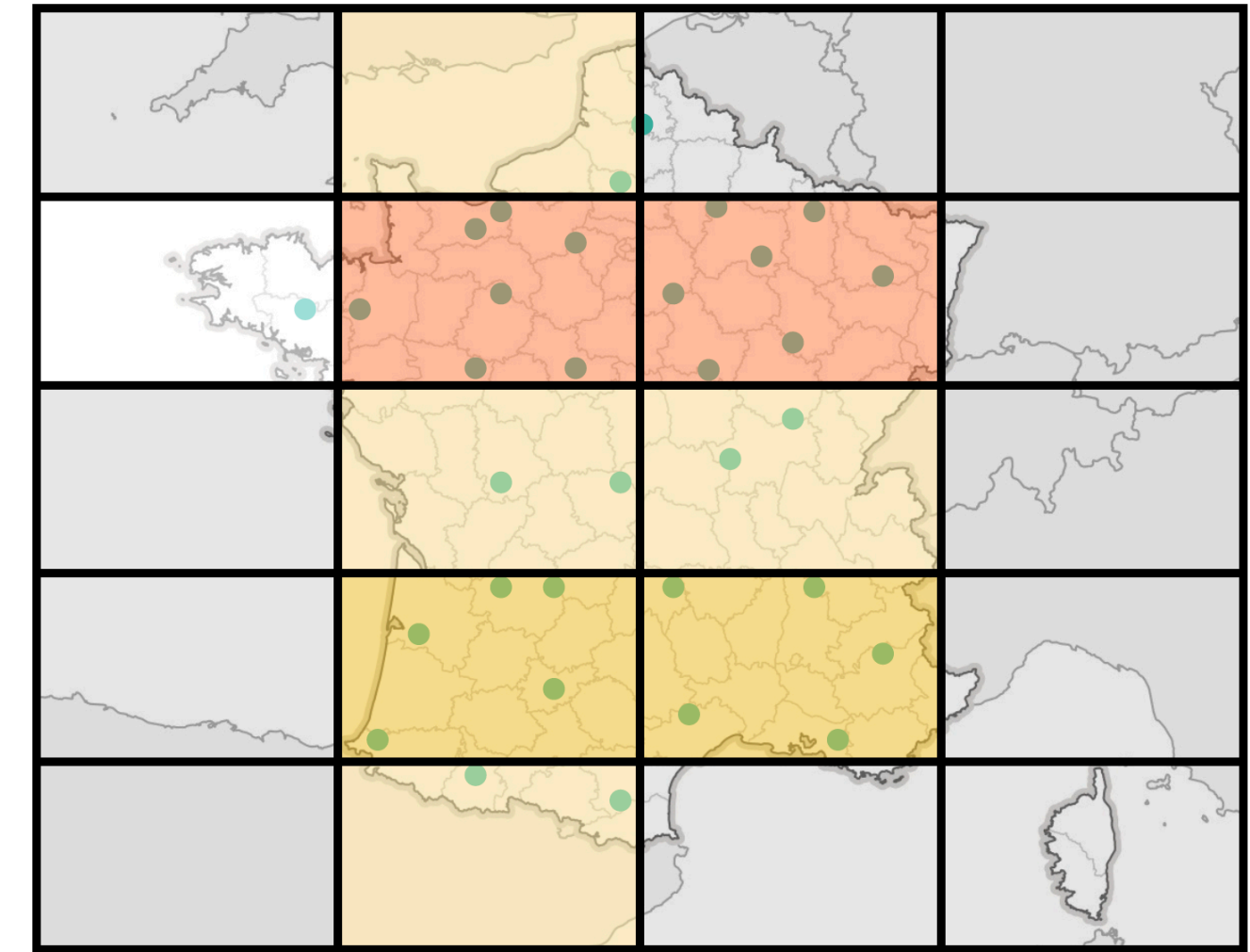
- To display the position where an event occurs you can use a map
- The documents will need to contain at least one geo point
 - type **geo_point** in Elasticsearch
- This geo point contains information about the latitude and longitude of the event

Visualization of Documents

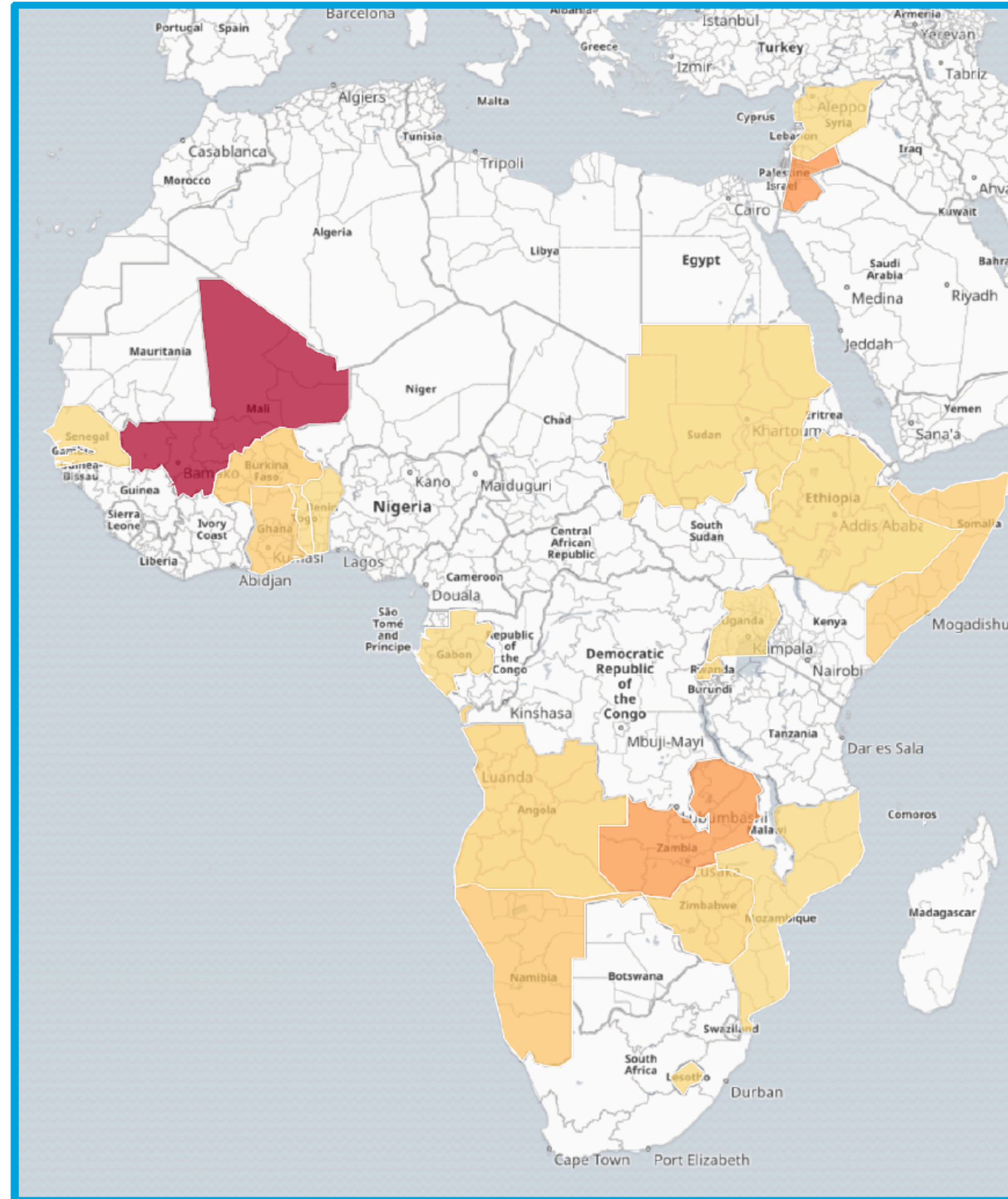


Grid Visualizations

- Geo points can also be displayed in a grid format
- A Grid is generated on the top of a Mercator map
- Granularity of the grid will depend on the zoom level
 - user's zoom action will yield higher precision on map

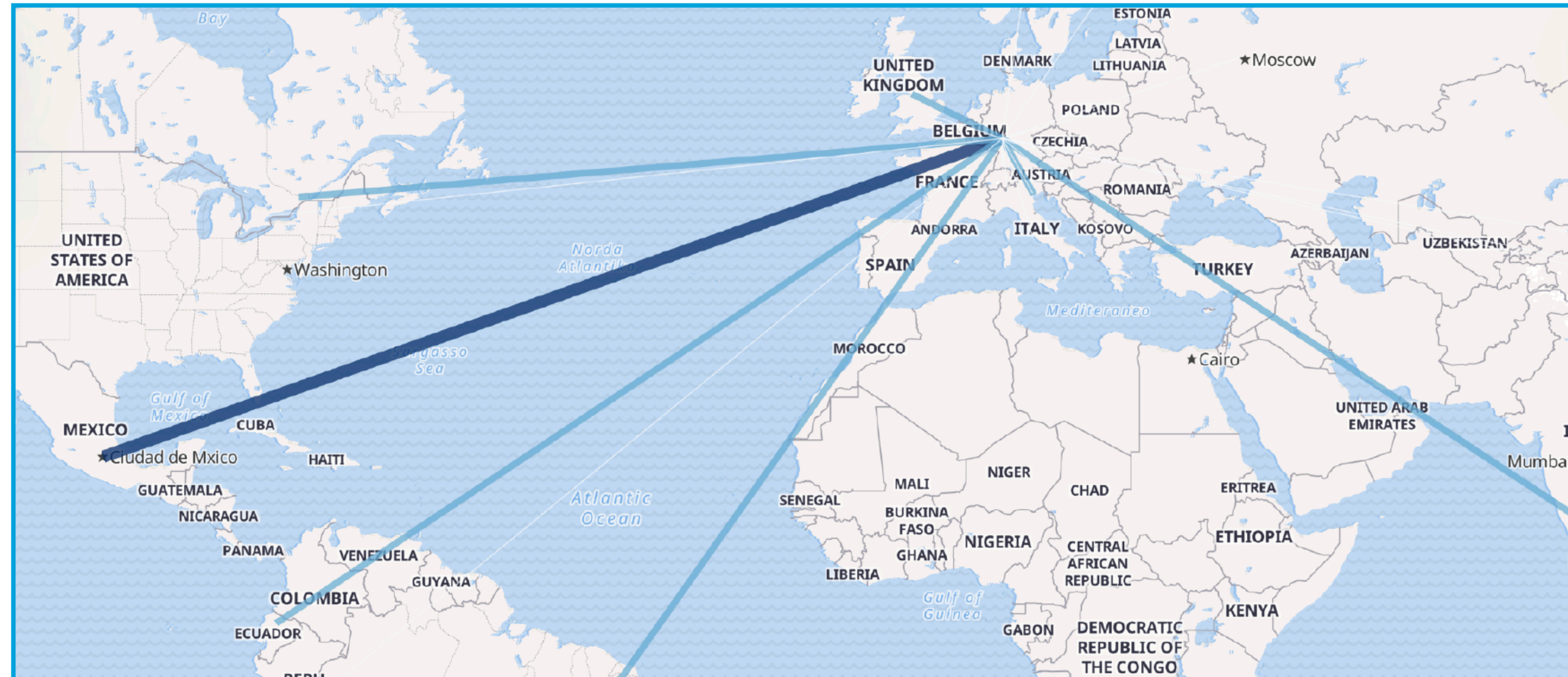


Displaying Countries



Point to Point Visualizations

- Point to point visualizations can be used in order to visualize the origin and destination of your data at the same time
- Very useful when visualizing a cyber attack for instance
- Data needs to have at least 2 geo points



Limitations

- Visualizations are interactive, it is possible:
 - to filter out some values
 - apply searches
 - select a time window
- However, there is still a lot that we cannot do with just visualizations:
 - compare different visualizations for the same time window
 - compare raw documents alongside other visualizations
 - drill down (filter) in one visualization (eg: map zooming) and analyze impact in another visualization

Data Comes in many Shapes

- The Elastic stack is used in many different contexts:



Search



Observe



Protect

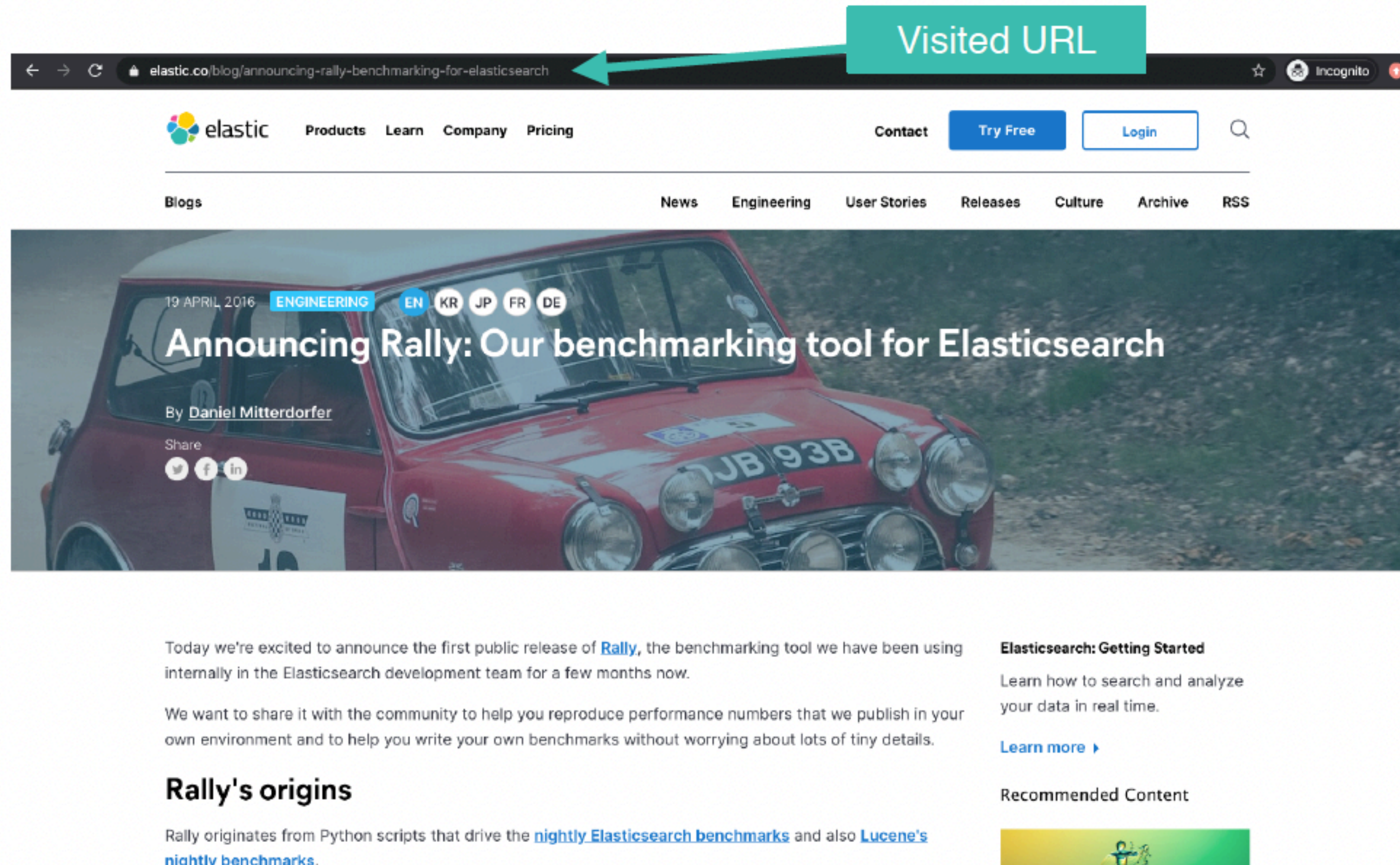
- Elasticsearch can store:
 - logs
 - metrics
 - security related data
 - business data
 - etc...

A Simple Example: Apache Access Logs

- Every time a user requests a resource from a website, this action is recorded in a log
- Every log entry contain:
 - visited URL
 - amount of bytes served
 - HTTP response code
 - information related to the client:
 - ip address
 - country/Region/city
 - information related to the browser and operating system

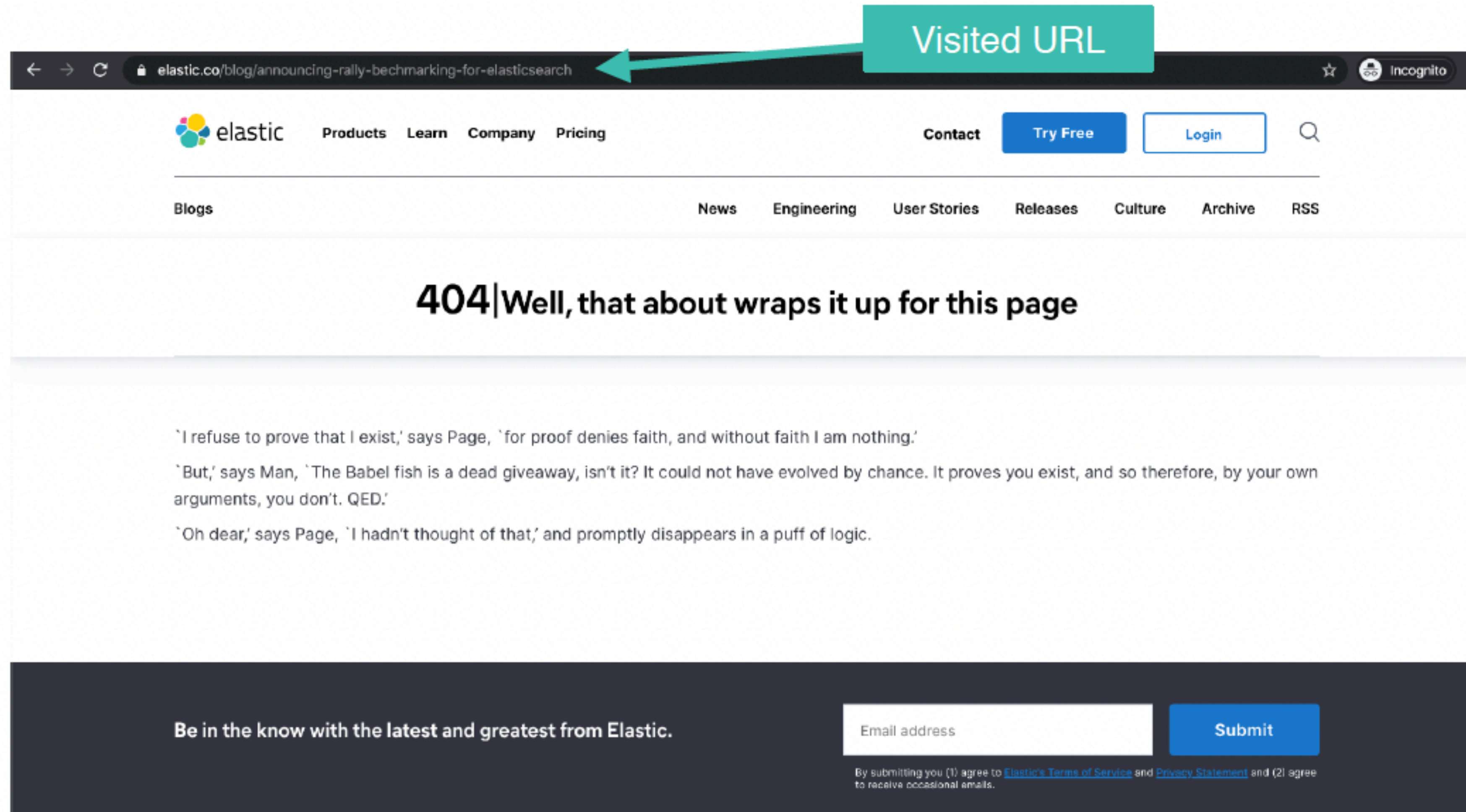
Correct URL

- The URL seems correct so the response code is likely to be **200**



Incorrect URL

- The URL is incorrect so the response code is a **404**



An Example of a Log Entry

```
{
  "@timestamp" : "2017-05-11T01:26:16.590Z",
  "request" : "/scripts/netcat-webserver",
  "agent" : "Mozilla/5.0 (compatible; EasouSpider; +http://www.easou.com/search/spider.html)",
  "bytes" : 40534,
  "geoip" : {
    "country_iso_code" : "US",
    "country_name" : "United States",
    "continent_name" : "NA",
    "location" : {
      "lon" : -77.4728,
      "lat" : 39.0481
    },
    "region_name" : "Virginia",
    "city_name" : "Ashburn"
  },
  "response" : 200,
  "verb" : "GET",
  "http_version" : "1.1",
  "clientip" : "192.168.215.50",
  "user_agent" : {
    "original" : "Mozilla/5.0 (compatible; EasouSpider; +http://www.easou.com/search/spider.html)",
    "name" : "EasouSpider",
    "device" : {
      "name" : "Spider"
    }
  }
}
```

The diagram illustrates a log entry with five annotations pointing to specific fields:

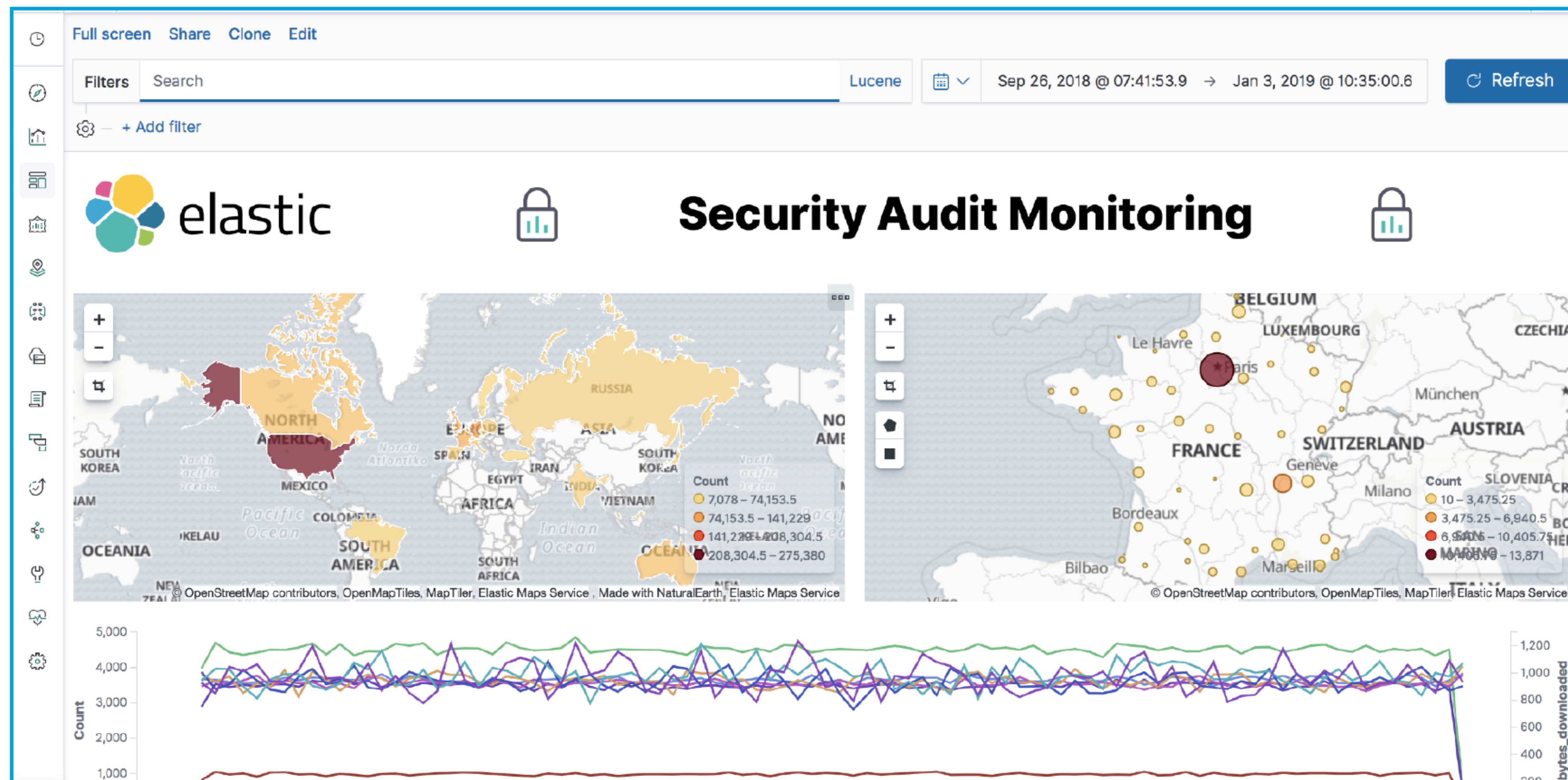
- Resource Accessed** points to the `"request"` field.
- GeoIP Information** points to the `"geoip"` object.
- Response Code** points to the `"response"` field.
- IP of the client** points to the `"clientip"` field.
- Browser Information** points to the `"user_agent"` object.

Using Sample Data

- When getting started with the Elastic stack, Elasticsearch will often not have any data stored in it
- Kibana provides a simple way to load sample data:
 - log data
 - flights dataset
 - ecommerce data
- Each of these sample datasets comes with:
 - visualizations
 - searches
 - Canvas
 - ...

Dashboards

- We can do all that and more with **Dashboards** :



Dashboards Overview

- Collection of visualizations and saved searches
- Requires at least one visualization or saved search
- To create a new dashboard
 - under Dashboard, "Create New Dashboard"
 - click "Add" to add visualizations and searches
 - resize, rearrange, reorder elements
 - save dashboard

Filters and Dashboards

- One of the main advantages of dashboards is the ability to dynamically drill down on data
- Different actions on visualizations dynamically creates different filters :
 - clicking on a bucket (*bar, pie-slice, etc.,*)
 - zooming on map (*double-click, rectangular select, etc.,*)
 - using input (this be covered later on)
 - using search bar query

Sharing Dashboard

- Once a dashboard is saved, you can share it
 - using a permalink
 - using an IFrame (embedded)
 - generating a raport (gold/platinuim license)
 - users will need Kibana access to view an embedded dashboard

Inline Frame (IFrame) and Permalink

- IFrames allow a Kibana dashboard (or visualization) to be embedded in an HTML document
 - copying an IFrame will provide html code wrapped in iframe tag
- Permalinks provides hyperlink that points directly to a Kibana dashboard or visualization
 - URL may be quiet long since it includes state information
 - URL encoded
 - URL shortening available as well
- Both IFrame et Permalink can generate link for either :
 - snapshot
 - saved object

Snapshot of a Dashboard

- To summarize the difference between snapshot and saved objects :
 - Snapshot :
 - the dashboard seen using the link (or IFrame) will not change if the underlying shared dashboard has been updated
 - Saved object:
 - the dashboard seen using the link ((or IFrame) will change if the underlying shared dashboard has been updated
- Take a snapshot of the visualization/dashboard in order to have it "frozen" and then share it with someone else.
 - in both cases the dashboard would show any data changes
 - but a snapshot will not reflect any changes to the dashboard itself

Kibana Access

- In order to access shared dashboard
 - you need Kibana access
 - appropriate user permissions
 - read-only
 - read, write and delete
- Elastic security can help manage
 - users
 - roles
 - permissions